KURSBUCH INFORMATIK

Stand: 14. Oktober 2026

Online-Version: https://glandl.github.io/Coursebook-maker-reise-grade-9/

KURSBUCH INFORMATIK

Willkommen zur **Maker-Reise**: In einem Schuljahr (2 Wochenstunden) durchläufst du fünf Projekte, die alle Kompetenzen des Lehrplans Informatik der 9.–11. Schulstufe abdecken.

Die fünf Projekte

1. **Code & Gadget** – Python-Grundlagen und dein erstes interaktives Gerät
2. **Wetterstation / IoT** – Sensoren, Netzwerke und Daten
3. **KI-Labor** – von Entscheidungsbäumen bis zu generativer KI
4. **Datendetektive** – SQL, Verschlüsselung, Privatsphäre
5. **Web-Portfolio** – deine eigene Projekt-Website, agil entwickelt

Begleitend führst du ein **Lernjournal** und arbeitest im Team mit **Kanban und Sprints**.

PROJEKTE

Die fünf Projekte der Maker-Reise. Jedes Projekt folgt demselben Rhythmus:

Kickoff & Anforderungen → Sprints (Bauen) → Review & Präsentation → Retrospektive + Journal-Abgabe

---

LERNJOURNAL

Das Lernjournal begleitet dich durch das ganze Jahr. Es ist dein Gedächtnis, dein Nachweis und später das Material für dein Web-Portfolio in P5. Es lebt als **Markdown-Dateien in deinem eigenen GitHub-Repository**. Jeder Eintrag ist ein Commit, deine Lehrkraft sieht den Fortschritt laufend, nicht erst am Projektende.

Aufbau des Repositorys

```
lernjournal/
├── README.md              ← Name, Klasse, Links zu deinen Einträgen
├── p1/
│   ├── woche-01.md        ← ein Eintrag pro Woche
│   ├── woche-02.md
│   └── …
├── p2/ … p5/
├── selbstlernen/
│   └── datenstrukturen.md ← Journalaufträge aus den Lernpfaden
└── images/                ← Screenshots und Fotos
```

Wie du das Repository anlegst und Dateien im Browser bearbeitest, steht unter Git & GitHub. Die Schreibweise erklärt der Markdown-Spickzettel.

Was hinein kommt

- **Ein Eintrag pro Woche** (Pflicht): Was habe ich gemacht, was hat funktioniert, wo bin ich hängen geblieben, was nehme ich mir vor?
- **Journalaufträge** aus den Projektwochen und den Selbstlern-Lernpfaden, jeweils mit Nummer und Titel als Überschrift.
- **Belege:** Screenshots, Fotos vom Aufbau, Code-Ausschnitte, Skizzen, Zustandsdiagramme.
- **Quellen:** Links, die dir geholfen haben, mit einem Satz, was du daraus mitgenommen hast.

Vorlage für einen Wocheneintrag

Datei `p1/woche-02.md`:

```
# P1 · Woche 2 · 2026-09-21

## Gemacht
- …

## Geklappt
- …

## Hängen geblieben
- …

## Nächster Schritt
- …

## Belege
![Screenshot schalter.py](../images/p1-w02-schalter.png)

## Quellen
- [Thonny-Anleitung](…) — hat mir beim Stopp-Button geholfen
```

Regeln

1. **Commit spätestens am Ende jeder Unterrichtswoche.** Die Commit-Nachricht sagt, was du geändert hast, z. B. `P1 Woche 2: Eintrag und Screenshot`.
2. **Lieber kurz und ehrlich als lang und schön.** Drei Sätze pro Abschnitt reichen.
3. **Belege im Ordner `images/`** ablegen, nicht verlinkt von fremden Diensten.
4. **Keine fremden Daten:** keine Fotos von Mitschüler:innen ohne Zustimmung, keine Passwörter, keine privaten Adressen. Das Repository ist privat, aber trotzdem.

Wann wird es angeschaut?

- **Laufend** über die Commit-Historie. Deine Lehrkraft sieht, wann du was geschrieben hast.
- Kurz bei jedem **Sprint-Review** (Zwischenpräsentation) im Team.
- Vollständig bei der **Journal-Abgabe** am Ende jedes Projekts. Abgabe heißt: der letzte Commit ist drin, es wird nichts eingesammelt.
- Die Journalaufträge aus den Selbstlern-Lernpfaden zählen als Nachweis, dass du den Lernpfad bearbeitet hast.

Warum das nicht lästig, sondern nützlich ist

Wer das Journal ehrlich führt, findet die Lösung vom letzten Mal in 30 Sekunden wieder, kann in der Retrospektive konkret sagen, was sich ändern soll, und hat für das Web-Portfolio in P5 den ganzen Inhalt schon fertig. Und die Commit-Historie zeigt dir selbst, wie viel du in einem Jahr geschafft hast.

P1 · CODE & GADGET

Dauer: ~7 Wochen (Woche 1–7) · **Team:** 2–3 Personen · **Sprache:** Python (PC) und MicroPython (Board)

Worum geht es?

Du lernst Python von Grund auf und baust am Ende ein **interaktives Gadget** auf einem Einplatinenrechner: ein Gerät, das auf Eingaben (Tasten, Sensoren) reagiert, sich in verschiedenen **Zuständen** befindet und über Ausgaben (LEDs, Display, Ton, Motor) antwortet.

Beispiele für Gadgets, die Teams gebaut haben oder bauen könnten:

- Reaktionsspiel mit zwei Tasten und Highscore
- Schrittzähler oder Würfel mit Beschleunigungssensor
- Nachtlicht, das auf Helligkeit und Bewegung reagiert
- Timer/Pomodoro-Uhr mit Display und Summer
- Alarmanlage mit Abstandssensor und Codeeingabe

Das Endprodukt

1. Ein **funktionierendes Gadget** auf micro:bit, Raspberry Pi Pico, ESP32 oder Arduino (deine Wahl, siehe MicroPython-Boards).
2. Ein **Zustandsdiagramm** deines Gadgets (welche Zustände gibt es, was löst Übergänge aus) und ein **Ablaufplan** der Hauptschleife.
3. Der **Quellcode** mit Kommentaren und Tests, im Team-Repository: `hardware.py` (board-spezifisch), `logik.py` (am PC testbar), `main.py`, dazu `README.md`, `ANFORDERUNGEN.md` und `TESTS.md`.
4. Eine **3-Minuten-Präsentation** beim Review in Woche 7.
5. Deine **Lernjournal-Einträge** (mindestens einer pro Woche).

Lernziele (Lehrplan)

Am Ende von P1 kannst du …

TB	Kompetenz (Lehrplan-Wortlaut)	Schulstufe
02 Algorithmen & Programmierung	Algorithmen in einer textbasierten Programmiersprache anhand einfacher Anwendungen umsetzen.	9
04 Systeme	Rechnersysteme mit Peripherie, Sensoren oder Aktoren sowie grundlegender Netzwerkfunktionalität konfigurieren und für lebensweltliche Aufgaben einsetzen. *(Netzwerk-Teil folgt in P2)*	10
06 Mensch-Maschine-Interaktion	Einfache interaktive Systeme bauen sowie deren Systemverhalten durch Variation von Eingaben und Rückmeldungen untersuchen und erklären.	10
09 Modellierung & Simulation	Reale Objekte oder Situationen zustandsbasiert und ablauforientiert abstrahieren und modellieren.	9

Ablauf

Woche	Phase	Inhalt
1	**Kickoff**	Maker-Reise, erste Python-Programme, Lernjournal auf GitHub anlegen (Markdown, erster Commit), Gadget-Ideen
2	Kickoff → Sprint 1	Python: Variablen, Verzweigungen, Schleifen · Team & Board wählen · Anforderungen an das Gadget · Kanban-Board anlegen

Woche	Phase	Inhalt
3	Sprint 1	Funktionen und Listen · Board in Betrieb nehmen: LED, Taste
4	Sprint 1 → Review	Sensoren/Aktoren anschließen · Zustandsdiagramm zeichnen · **Sprint-Review 1**
5	Sprint 2	Zustandsautomat in Code umsetzen · Verhalten durch Eingaben variieren
6	Sprint 2	Feinschliff, Testen, Doku · Präsentation vorbereiten
7	**Review & Retro**	Präsentationen · Retrospektive · Journal-Abgabe

Werkzeuge

- Python & Thonny — ab Woche 1
- MicroPython-Boards — ab Woche 3
- Wokwi & Browser-Simulatoren — Fallback ohne Hardware
- Git & GitHub und Markdown — ab Woche 1 für das Lernjournal
- Lernjournal — von Anfang an

Wochen

- Woche 1 · Kickoff
- Woche 2 · Team, Anforderungen, Kanban
- Woche 3 · Funktionen, Listen, erstes Board
- Woche 4 · Sensoren, Zustandsdiagramm, Sprint-Review
- Woche 5 · Zustandsautomat und Experiment
- Woche 6 · Testen, Code-Review, Präsentation
- Woche 7 · Review und Retrospektive {class="children children-type-tree children-sort-weight"}

WOCHE 1 · KICKOFF

Heute: Du lernst die Maker-Reise kennen, schreibst deine ersten Python-Programme, legst dein Lernjournal auf GitHub an und schreibst den ersten Eintrag in Markdown.

Was du am Ende der Stunde hast

- Thonny läuft auf deinem Rechner und du hast zwei Programme ausgeführt und verändert.
- Du weißt, was ein **Zustand** ist und hast ein Gadget mit Zuständen simuliert.
- Du hast ein GitHub-Konto und ein Lernjournal-Repository mit deinem ersten **Commit**.
- Dein erster Journal-Eintrag steht, in Markdown, mit mindestens zwei Gadget-Ideen.

1. Die Maker-Reise (10 min)

Ein Schuljahr, fünf Projekte, ein Lernjournal. Schau dir die Startseite an: Jedes Projekt läuft nach demselben Rhythmus **Kickoff → Sprints → Review → Retro**. In P1 baust du dein erstes interaktives Gerät. Was am Ende herauskommen soll, steht im Projektbrief. Die Boards, auf denen das laufen wird, gehen heute durch die Klasse.

2. Werkzeug-Check: Thonny (15 min)

Öffne Thonny. Wenn es auf deinem Gerät noch fehlt, folge der Setup-Anleitung. Notfalls geht auch der Browser: python.microbit.org hat eine Python-Konsole und einen Simulator.

Tippe unten in der **Shell** (das Fenster mit `>>>`) und drücke Enter:

```
print("Hallo Maker-Reise!")
```

Wenn der Text erscheint, bist du startklar.

3. Zwei Programme (30 min)

Lege für jedes Programm eine neue Datei an (**Datei → Neu**), speichere sie unter dem angegebenen Namen und starte sie mit dem grünen Play-Button (oder **F5**).

Programm 1 · `hallo.py` — Eingabe und Ausgabe

```
name = input("Wie heißt du? ")
print("Hallo", name, "— willkommen zur Maker-Reise!")
```

Ändere es: Frag zusätzlich nach dem Lieblingsgerät und gib beides in einem Satz aus.

Programm 2 · `schalter.py` — ein Gadget mit zwei Zuständen

Fast jedes Gerät hat **Zustände**: Eine Lampe ist AUS oder AN. Ein Tastendruck wechselt den Zustand. Genau das simulieren wir hier in Text, bevor wir es in ein paar Wochen mit echter LED und Taste bauen.

```
# Ein Gadget mit zwei Zuständen: AUS und AN
zustand = "AUS"

while True:
    print("Lampe ist", zustand)
    taste = input("Taste drücken [Enter], 'q' zum Beenden: ")
    if taste == "q":
        break
    if zustand == "AUS":
        zustand = "AN"
    else:
        zustand = "AUS"

print("Gadget ausgeschaltet.")
```

Ändere es:

1. Zähle mit, wie oft die Taste gedrückt wurde, und gib die Zahl am Ende aus.
2. Die Lampe soll nach dem fünften Tastendruck von selbst ausgehen („Energiesparmodus").

Tipp zu 1.

Lege vor der Schleife `druecke = 0` an und erhöhe die Variable in der Schleife mit `druecke = druecke + 1`.

Mach einen Screenshot vom laufenden Programm. Den brauchst du gleich für das Journal.

Wenn du schnell bist (oder zu Hause) · `ampel.py` — drei Zustände im Kreis

```
zustand = "ROT"
schaltvorgaenge = 0

while schaltvorgaenge < 6:
    print("Ampel:", zustand)
    input("Weiter mit [Enter]")
    if zustand == "ROT":
        zustand = "GRÜN"
    elif zustand == "GRÜN":
        zustand = "GELB"
    else:
        zustand = "ROT"
    schaltvorgaenge = schaltvorgaenge + 1

print("Fertig nach", schaltvorgaenge, "Schaltvorgängen.")
```

Ändere es: Eine echte Ampel geht ROT → ROT-GELB → GRÜN → GELB → ROT. Baue den vierten Zustand ein. Zeichne danach die Zustände als Kreise und die Übergänge als Pfeile auf Papier. Das ist dein erstes **Zustandsdiagramm**.

Ausblick: dasselbe Programm auf dem Board

Ab Woche 3 läuft dein Schalter auf echter Hardware. Der Kern bleibt gleich, nur `input` und `print` werden durch Taste und LED ersetzt. Du musst das heute nicht verstehen, nur sehen, dass es dieselbe Idee ist.

micro:bit

```
from microbit import *

zustand = "AUS"
while True:
    if button_a.was_pressed():
        if zustand == "AUS":
            zustand = "AN"
            display.show(Image.HAPPY)
        else:
            zustand = "AUS"
            display.clear()
```

Raspberry Pi Pico

```
from machine import Pin
import time

led = Pin("LED", Pin.OUT)              # Onboard-LED
taste = Pin(14, Pin.IN, Pin.PULL_UP)   # Taste zwischen GP14 und GND

zustand = "AUS"
while True:
    if taste.value() == 0:             # gedrückt
        zustand = "AN" if zustand == "AUS" else "AUS"
        led.value(1 if zustand == "AN" else 0)
        time.sleep(0.3)                # Entprellen
```

ESP32

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)                  # Onboard-LED (meist GPIO 2)
taste = Pin(4, Pin.IN, Pin.PULL_UP)    # Taste zwischen GPIO 4 und GND

zustand = "AUS"
while True:
    if taste.value() == 0:
        zustand = "AN" if zustand == "AUS" else "AUS"
        led.value(1 if zustand == "AN" else 0)
        time.sleep(0.3)
```

Arduino

Ein **Arduino Nano ESP32** läuft mit MicroPython, dann gilt der ESP32-Code (LED-Pin anpassen). **UNO R3** und **UNO R4** können kein MicroPython: nimm dafür den Wokwi-Simulator mit einem virtuellen ESP32 oder den C++-Anhang.

4. Dein Lernjournal auf GitHub (25 min)

Dein Journal ist eine Sammlung von **Markdown-Dateien in einem GitHub-Repository**. Jede Änderung speicherst du als **Commit**. So siehst du selbst und deine Lehrkraft, wie es Woche für Woche wächst.

4a · Konto und Repository (15 min)

Folge Teil 1 und Teil 2 der Anleitung Git & GitHub. Am Ende hast du ein Repository `lernjournal` mit einer `README.md`.

4b · Markdown in fünf Minuten (5 min)

Öffne den Markdown-Spickzettel. Du brauchst heute nur: `#` für Überschriften, `-` für Listen, `**fett**`, und wie man ein Bild einbindet.

4c · Erster Commit (5 min)

Bearbeite die `README.md` im Browser (Stift-Symbol), so dass sie so aussieht, mit deinen Angaben:

```
# Lernjournal von <Vorname>

Klasse: <Klasse> · Schuljahr 2026/27 · Maker-Reise

## Einträge
- [P1 Woche 1](p1/woche-01.md)
```

Reiter **Preview** anschauen, dann **Commit changes** mit der Nachricht `README: Name und Klasse`. Dein erster Commit.

5. Journalauftrag 1 (15 min)

Lege im Repository die Datei `p1/woche-01.md` an (**Add file** → **Create new file**, der Schrägstrich erzeugt den Ordner) und schreib deinen ersten Eintrag:

Journalauftrag 1 · Startpunkt

```
# P1 · Woche 1 · <Datum>

## Erwartungen
Was erwartest du dir von der Maker-Reise? Was kannst du schon
(Programmieren, Elektronik, Basteln)?

## Gadget-Ideen
Mindestens zwei Geräte aus deinem Alltag, die du gern bauen würdest.
Zu jeder Idee: Eingaben (was misst oder drückt man), Ausgaben
(was leuchtet, piept, bewegt sich), Zustände.

## Geklappt / hängen geblieben
Was hat heute funktioniert, wo bist du stecken geblieben?

## Beleg
Screenshot von schalter.py: in den Ordner images/ hochladen und
hier einbinden: ![schalter.py](../images/p1-w01-schalter.png)
```

Commit-Nachricht: `P1 Woche 1: erster Eintrag`. Wenn der Screenshot-Upload heute nicht mehr klappt, holst du ihn bis nächste Woche nach.

Falls du Inspiration für die Gadget-Ideen brauchst: Das kann ein micro:bit.

▶ *Video: https://www.youtube.com/watch?v=SF_Ym8fWTPU*

Check dich selbst

Was ist der Unterschied zwischen `print` und `input`?

`print` gibt Text aus. `input` zeigt einen Text an, wartet auf eine Eingabe und liefert sie als Text zurück, den du in einer Variable speichern kannst.

Was passiert in `schalter.py`, wenn du die Zeile `if taste == "q": break` löschst?

Die Schleife läuft endlos. Du kannst das Programm nur noch mit dem Stopp-Button (oder Strg+C in der Shell) beenden.

Wie viele Zustände hat ein Fahrradlicht mit den Modi Aus, Dauerlicht, Blinken?

Drei. Ein Tastendruck geht im Kreis: Aus → Dauerlicht → Blinken → Aus. Das ist genau die Struktur von `ampel.py`.

Was ist ein Commit, und warum braucht er eine Nachricht?

Ein Commit ist ein gespeicherter Stand deines Repositorys mit Zeitpunkt und Autor:in. Die Nachricht sagt, was sich geändert hat, damit du (und deine Lehrkraft) die Geschichte des Journals lesen können, ohne jede Datei zu öffnen.

Bis nächste Woche

- Journalauftrag 1 fertigstellen und committen (Screenshot nicht vergessen).
- `ampel.py` ausprobieren, falls du heute nicht dazu gekommen bist.
- Thonny zu Hause installieren, falls du ein eigenes Gerät nutzt (Anleitung).
- Video zum Nachschauen, wenn heute etwas zu schnell ging:

▶ *Video: https://www.youtube.com/watch?v=oxXAb8IikHM*

WOCHE 2 · TEAM, ANFORDERUNGEN, KANBAN

Heute: Du lernst die drei Grundbausteine jedes Programms kennen (Variablen, Verzweigungen, Schleifen), findest dein Team, wählt euer Board, schreibt auf, was euer Gadget können muss, und legt ein Kanban-Board an. Ab heute läuft **Sprint 1** (Woche 2–4).

Was du am Ende der Stunde hast

- Du kannst Eingaben in Zahlen umwandeln, mit `if`/`elif`/`else` entscheiden und mit `while` und `for` wiederholen.
- Du bist in einem Team (2–3 Personen) und ihr habt ein Board gewählt.
- Euer Team-Repository enthält eine Datei `ANFORDERUNGEN.md` mit User Stories.
- Euer Kanban-Board steht, die Karten für Sprint 1 sind ausgewählt.
- Journalauftrag 2 ist committet.

1. Rückblick (5 min)

Zeig deiner Sitznachbarin oder deinem Sitznachbarn deine Gadget-Ideen aus Journalauftrag 1. Welche Idee hat die klarsten **Eingaben**, **Ausgaben** und **Zustände**?

2. Python-Werkstatt (30 min)

2a · Variablen und Datentypen · `thermostat.py`

Eine **Variable** ist ein Name für einen Wert. Der Wert hat einen **Datentyp**: `int` (ganze Zahl), `float` (Kommazahl), `str` (Text), `bool` (`True`/`False`).

`input()` liefert **immer Text**. Mit Text kann man nicht rechnen, deshalb wandelst du ihn um: `int(...)` oder `float(...)`.

```
# Ein Thermostat entscheidet, ob geheizt wird
temperatur = float(input("Raumtemperatur in °C: "))
wunsch = 21.0

if temperatur < wunsch - 1:
    print("Heizung AN")
elif temperatur > wunsch + 1:
    print("Heizung AUS, Fenster auf?")
else:
    print("Passt. Heizung bleibt, wie sie ist.")

print("Abweichung:", round(temperatur - wunsch, 1), "°C")
```

Probier aus: Gib `18`, `21` und `23.5` ein. Dann gib `warm` ein. Lies die Fehlermeldung: Welche Zeile, welcher Fehler?

Ändere es:

1. Frag die Wunschtemperatur auch mit `input` ab.
2. Unter 5 °C soll zusätzlich `Frostschutz!` erscheinen.

Tipp zu 2.

Die Reihenfolge der Bedingungen zählt: Python nimmt den **ersten** zutreffenden Zweig. Prüfe deshalb `temperatur < 5` **vor** `temperatur < wunsch - 1`, oder schreib ein eigenes `if` vor den ganzen Block.

2b · Vergleichen und verknüpfen

Ausdruck	Bedeutung
`a == b`, `a != b`	gleich, ungleich (ein `=` allein ist eine **Zuweisung**)
`a < b`, `a <= b`, `a > b`, `a >= b`	kleiner, kleiner-gleich, …
`x and y`	beides wahr
`x or y`	mindestens eines wahr
`not x`	Gegenteil

2c · Schleifen · `countdown.py`

`while` wiederholt, **solange** eine Bedingung gilt (kennst du aus `schalter.py`). `for` wiederholt **eine bestimmte Anzahl** von Malen oder für jedes Element einer Folge.

```
import time

start = int(input("Countdown ab: "))

for sekunde in range(start, 0, -1):
    print(sekunde)
    time.sleep(1)

print("Los!")

# Blinkmuster: dreimal kurz, dann lang
for i in range(3):
    print("*", end=" ")
    time.sleep(0.3)
print("******")
```

`range(start, 0, -1)` zählt von `start` rückwärts bis **vor** 0, also bis 1. `range(3)` liefert 0, 1, 2.

Ändere es:

1. Bei den letzten drei Sekunden soll zusätzlich `Piep!` erscheinen.
2. Frag so lange nach dem Startwert, bis eine Zahl zwischen 1 und 60 eingegeben wird.

<details>
<summary>Tipp zu 2.</summary>

```
start = 0
while start < 1 or start > 60:
    start = int(input("Countdown ab (1–60): "))
```

</details>

<details>
<summary>Wenn du schnell bist · `wuerfel.py`</summary>

Ein elektronischer Würfel ist ein beliebtes Gadget. Schreib ein Programm, das bei jedem Enter eine Zufallszahl von 1 bis 6 ausgibt und bei einer **Sechs** `Nochmal!` meldet. Nach 10 Würfen zeigt es, wie oft eine Sechs dabei war.

```
import random
zahl = random.randint(1, 6)   # Zufallszahl von 1 bis 6
```

</details>

Zum Nachschauen (Verzweigungen, dann beide Schleifenarten):

▶ *Video: https://www.youtube.com/watch?v=Kc-JmsKLPjs*

▶ *Video: https://www.youtube.com/watch?v=cSkP6rYQ3U0*

3. Team und Board (15 min)

Teams: 2–3 Personen. Bildet Teams rund um eine gemeinsame Gadget-Idee, nicht nur um Freundschaften. Die Lehrkraft hilft, wenn jemand übrig bleibt.

Board: Alle vier sind gleichwertig. Die Wahl hängt davon ab, was euer Gadget braucht.

Board	Stärken	Gut für	Achtung
micro:bit (V2)	Tasten, 5×5-LED-Anzeige, Lautsprecher, Mikrofon, Licht-, Lage- und Temperatursensor schon eingebaut. Kein Stecken nötig.	schneller Start, Spiele, Schrittzähler, Würfel	wenig freie Anschlüsse, kein WLAN
Raspberry Pi Pico (W)	günstig, viele Anschlüsse, analoge Eingänge	Nachtlicht, Alarmanlage, alles mit Steckbrett	Sensoren und LEDs selbst anschließen. Nur die **Pico W** hat WLAN.
ESP32	WLAN und Bluetooth eingebaut, schnell	alles, was später ins Netz soll	Pinbelegung je nach Modell unterschiedlich
Arduino	robust, weit verbreitet	wenn das Board schon da ist	**UNO R3** und **R4** können kein MicroPython (der **Nano ESP32** schon): dann Wokwi mit virtuellem ESP32

Blick nach vorn

In **P2 (Wetterstation)** schickt ihr Messwerte über WLAN. Mit **ESP32** oder **Pico W** könnt ihr euer Board dort weiterverwenden. Mit micro:bit ist P1 am einfachsten, für P2 braucht ihr dann ein zweites Board.

Team-Repository anlegen: Eine Person legt auf GitHub ein **privates** Repository `p1-gadget-<teamname>` an (mit README), wie in Git & GitHub Teil 2 „Selbst anlegen". Unter **Settings → Collaborators** die anderen Teammitglieder und die Lehrkraft einladen. Code, Anforderungen und Zustandsdiagramm liegen hier, dein **persönliches** Journal bleibt in deinem `lernjournal`.

4. Anforderungen: Was muss euer Gadget können? (25 min)

Bevor ihr baut, schreibt ihr auf, **was** das Gadget tun soll, nicht **wie**. Das verhindert, dass nach drei Wochen jedes Teammitglied an einem anderen Gerät baut.

4a · User Stories

Eine **User Story** beschreibt eine Anforderung aus Sicht einer Person, die das Gadget benutzt:

> ***Als*** *<Rolle>* ***möchte ich*** *<Funktion>,* ***damit*** *<Nutzen>.*

Zu jeder Story gehören **Akzeptanzkriterien**: überprüfbare Sätze, an denen ihr beim Review erkennt, ob die Story fertig ist.

Beispiel Nachtlicht:

```
### US1 · Licht bei Dunkelheit (Muss)
Als Kind möchte ich, dass das Nachtlicht von selbst angeht, wenn es dunkel wird,
damit ich nachts nicht im Finstern aufstehen muss.

- [ ] Bei abgedeckter Hand über dem Sensor geht die LED innerhalb von 1 s an.
- [ ] Bei Tageslicht bleibt die LED aus.

### US2 · Ausschalten (Muss)
Als Elternteil möchte ich das Nachtlicht mit einer Taste ganz ausschalten,
damit es tagsüber keinen Strom braucht.

- [ ] Ein Tastendruck schaltet zwischen AUS und AUTOMATIK um.
- [ ] Im Zustand AUS leuchtet die LED nie, auch nicht bei Dunkelheit.

### US3 · Sanftes Ausgehen (Kann)
Als Kind möchte ich, dass das Licht langsam dunkler wird, damit ich nicht erschrecke.
```

Muss / Soll / Kann sagt, wie wichtig eine Story ist. Für Sprint 1 nehmt ihr nur **Muss**-Stories.

Woran erkenne ich eine gute User Story?

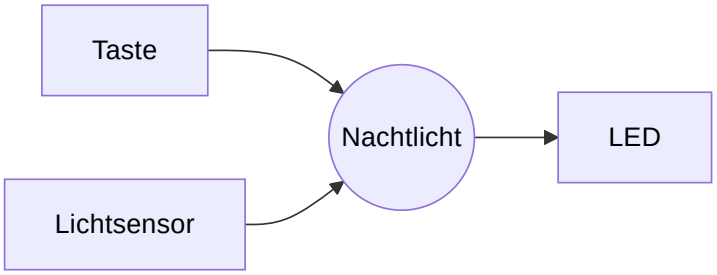
- Sie beschreibt **ein** Verhalten, das man vorführen kann.
- Sie sagt nichts über Code oder Pins („Als Nutzer möchte ich eine `while`-Schleife" ist keine Story).
- Die Akzeptanzkriterien kann eine fremde Person prüfen, ohne euch zu fragen: „innerhalb von 1 s" statt „schnell".

▶ *Video: https://www.youtube.com/watch?v=vYnprnrTWwU*

4b · Eingaben → Gadget → Ausgaben

Zeichnet dazu eine einfache Skizze: Was geht hinein, was kommt heraus? GitHub zeigt das sogar direkt als Grafik an, wenn ihr es als **Mermaid**-Block in eure Markdown-Datei schreibt:

````
```mermaid
flowchart LR
    T[Taste] --> G((Nachtlicht))
    L[Lichtsensor] --> G
    G --> LED[LED]
```
````



Auftrag im Team: Legt im Team-Repository `ANFORDERUNGEN.md` an mit

1. einem Satz, was euer Gadget ist,
2. der Skizze Eingaben → Gadget → Ausgaben,
3. **3–5 User Stories** mit Priorität und Akzeptanzkriterien.

5. Kanban-Board (10 min)

Ein **Kanban-Board** zeigt auf einen Blick, wer woran arbeitet und was fertig ist.

Backlog	Sprint 1	In Arbeit	Fertig
alle Stories und Aufgaben	was ihr bis zum Review in Woche 4 schaffen wollt	was gerade jemand macht (mit Name)	Akzeptanzkriterien erfüllt

Regeln:

1. **Eine Karte = eine Aufgabe**, klein genug für eine Stunde. Große Stories zerlegt ihr: „LED anschließen", „Sensorwert ausgeben", „Schwellwert finden".
2. **Höchstens eine Karte pro Person** in „In Arbeit" (WIP-Limit). Erst fertig machen, dann Neues anfangen.
3. **Fertig** heißt: die Akzeptanzkriterien sind erfüllt, nicht „fast".
4. Jede Stunde beginnt mit einem **Stand-up** (3 min) vor dem Board: Was habe ich gemacht? Was mache ich heute? Wo hänge ich?

Wo das Board lebt, entscheidet ihr: **Papier/Haftnotizen** (Foto am Stundenende ins Repository) oder digital als **GitHub Project** im Team-Repository (Reiter **Projects** → **New project** → **Board**).

▶ *Video: https://www.youtube.com/watch?v=P513uNckJF0*

6. Journalauftrag 2 (10 min)

Lege in **deinem** Lernjournal `p1/woche-02.md` an und verlinke es in der `README.md`.

Journalauftrag 2 · Team und Plan

```
# P1 · Woche 2 · <Datum>

## Team und Gadget
Mit wem bist du im Team? Was baut ihr (ein Satz)?
Welches Board, und warum dieses und kein anderes?

## Meine Aufgaben in Sprint 1
Welche Karten vom Kanban-Board übernimmst du?

## Python
Ein Ding, das ich heute verstanden habe (mit einem Codebeispiel).
Ein Ding, das mir noch unklar ist.

## Beleg
Foto vom Kanban-Board oder Screenshot von thermostat.py mit deiner Änderung.
Link zu ANFORDERUNGEN.md im Team-Repository.
```

Commit-Nachricht: `P1 Woche 2: Team, Plan, Python`.

Quiz

Quiz · Woche 2

1. Was passiert bei diesem Code, wenn du `15` eingibst?

```
alter = input("Alter? ")
print(alter + 1)
```

a) Es wird `16` ausgegeben.

b) Es wird `151` ausgegeben.

c) Es gibt einen Fehler.

d) Es wird `15` ausgegeben.

2. Welche Zahlen gibt `for i in range(3): print(i)` aus?

a) 1, 2, 3

b) 0, 1, 2

c) 0, 1, 2, 3

d) 3, 2, 1

3. `temperatur = 22`. Was gibt das Programm aus?

```
if temperatur > 25:
    print("heiß")
elif temperatur > 18:
    print("angenehm")
else:
    print("kalt")
```

a) heiß

b) angenehm

c) kalt

d) angenehm und kalt

4. Welche ist eine gute User Story?

a) Das Gadget soll gut sein.

b) Als Programmierer möchte ich eine `while`-Schleife verwenden.

c) Wir bauen einen Summer an Pin 15.

d) Als Läuferin möchte ich meine Schritte sehen, damit ich weiß, ob ich mein Tagesziel erreicht habe.

5. Wozu dient das WIP-Limit („höchstens eine Karte pro Person in Arbeit")?

a) Damit Aufgaben fertig werden, statt dass vieles halb angefangen herumliegt.

b) Damit niemand zu viel arbeitet.

c) Damit die Lehrkraft weniger kontrollieren muss.

d) Damit das Board übersichtlich aussieht.

Bis nächste Woche

- `ANFORDERUNGEN.md` im Team fertigstellen, Kanban-Board mit Karten für Sprint 1 füllen.
- Journalauftrag 2 committen.
- Wer Hardware zu Hause hat: MicroPython-Boards durchlesen und das Board mit Thonny verbinden. Ohne Hardware: Wokwi oder python.microbit.org öffnen.

WOCHE 3 · FUNKTIONEN, LISTEN, ERSTES BOARD

Heute: Du verpackst Code in **Funktionen**, speicherst viele Werte in **Listen** und bringst euer Board zum Leben: LED blinkt, Taste schaltet.

Was du am Ende der Stunde hast

- Du kannst eine Funktion mit Parametern und Rückgabewert schreiben und aufrufen.
- Du kannst eine Liste anlegen, erweitern, durchlaufen und mit dem Index auf Elemente zugreifen.
- Euer Board ist mit Thonny verbunden, eine LED blinkt und eine Taste schaltet sie um.
- Der Board-Code liegt im Team-Repository, das Kanban-Board ist aktuell.

1. Stand-up (5 min)

Vor dem Kanban-Board, jede Person höchstens eine Minute: **Was habe ich gemacht? Was mache ich heute? Wo hänge ich?** Karten verschieben, Namen draufschreiben.

2. Funktionen (15 min)

Eine **Funktion** ist ein benannter Codeblock. Du schreibst ihn einmal mit `def` und rufst ihn beliebig oft auf. Werte gibst du als **Parameter** hinein, mit `return` gibt die Funktion ein Ergebnis zurück.

`schalter_funktionen.py`

Das ist `schalter.py` aus Woche 1, aufgeräumt: Die Logik („was kommt nach AUS?") und die Ausgabe sind jetzt getrennte Funktionen.

```
def umschalten(zustand):
    """Liefert den neuen Zustand nach einem Tastendruck."""
    if zustand == "AUS":
        return "AN"
    return "AUS"


def anzeigen(zustand, druecke):
    symbol = "💡" if zustand == "AN" else "⚫"
    print(symbol, "Lampe ist", zustand, "| Tastendrücke:", druecke)


zustand = "AUS"
druecke = 0
anzeigen(zustand, druecke)

while True:
    taste = input("[Enter] drücken, 'q' zum Beenden: ")
    if taste == "q":
        break
    zustand = umschalten(zustand)
    druecke = druecke + 1
    anzeigen(zustand, druecke)
```

Warum der Aufwand? In ein paar Minuten ersetzt du `input` durch eine echte Taste und `print` durch eine echte LED. Die Funktion `umschalten` bleibt **unverändert**. Getrennte Logik kann man außerdem ohne Hardware testen (das machen wir in Woche 6).

Ändere es: Füge einen dritten Zustand `BLINKEN` hinzu: AUS → AN → BLINKEN → AUS. Du musst nur `umschalten` ändern (und das Symbol in `anzeigen`).

3. Listen (15 min)

Eine **Liste** speichert mehrere Werte in einer Variable, in fester Reihenfolge.

```
zustaende = ["ROT", "ROT-GELB", "GRÜN", "GELB"]

print(zustaende[0])        # ROT      – Index beginnt bei 0
print(zustaende[-1])       # GELB     – von hinten
print(len(zustaende))      # 4

for z in zustaende:
    print("Ampel:", z)
```

`ampel_liste.py` — Zustände im Kreis

```
import time

ZUSTAENDE = ["ROT", "ROT-GELB", "GRÜN", "GELB"]
DAUER = [3, 1, 3, 1]              # Sekunden pro Zustand

i = 0
for schritt in range(8):
    print("Ampel:", ZUSTAENDE[i])
    time.sleep(DAUER[i])
    i = (i + 1) % len(ZUSTAENDE)   # nach dem letzten wieder zum ersten
```

`%` ist der **Rest** der Division: `4 % 4` ist 0, darum springt der Index vom letzten Element zurück auf 0. Namen in GROSSBUCHSTABEN bedeuten: Dieser Wert ändert sich im Programm nicht (Konstante).

`messwerte.py` — Werte sammeln und auswerten

Sensoren liefern laufend Werte. Die sammelst du in einer Liste:

```
messwerte = []                    # leere Liste

while True:
    eingabe = input("Messwert (leer = fertig): ")
    if eingabe == "":
        break
    messwerte.append(float(eingabe))

if len(messwerte) > 0:
    print("Anzahl:", len(messwerte))
    print("Kleinster:", min(messwerte), "Größter:", max(messwerte))
    print("Mittelwert:", round(sum(messwerte) / len(messwerte), 2))
```

Ändere es: Schreib eine Funktion `mittelwert(werte)`, die den Mittelwert zurückgibt, und verwende sie.

Lösung

```
def mittelwert(werte):
    return sum(werte) / len(werte)
```

▶ *Video: https://www.youtube.com/watch?v=B1mMAieycPY*

▶ *Video: https://www.youtube.com/watch?v=R7lZTvC9UUU*

4. Board in Betrieb nehmen (45 min)

4a · Verbinden (10 min)

1. Board per USB anschließen.
2. Thonny: **Werkzeuge** → **Optionen** → **Interpreter**, das passende MicroPython wählen (bei micro:bit: „MicroPython (BBC micro:bit)"). Ist noch kein MicroPython auf dem Board, bietet Thonny unten rechts **Install or update MicroPython** an.
3. In der Shell `print("Hallo Board")` eintippen. Die Antwort kommt jetzt **vom Board**, nicht vom PC.

Details, Treiber und Fehlerbehebung je Board: MicroPython-Boards. **Kein Board zur Hand?** Wokwi (Pico, ESP32) oder python.microbit.org (micro:bit mit Simulator) funktionieren mit demselben Code.

⚠ Strom aus beim Umstecken

Bauteile nur anschließen oder umstecken, wenn das USB-Kabel **abgezogen** ist. LEDs immer mit **Vorwiderstand** (220 Ω). Eine LED hat eine Richtung: das **lange Bein** (+) zeigt zum Pin, das kurze zu GND.

4b · Blinken (10 min)

micro:bit

Keine Verkabelung nötig. Die LED in der Mitte der 5×5-Anzeige hat die Koordinaten (2, 2).

```
from microbit import *

for i in range(10):
    display.set_pixel(2, 2, 9)   # Helligkeit 0–9
    sleep(500)                   # Millisekunden
    display.set_pixel(2, 2, 0)
    sleep(500)
```

Raspberry Pi Pico

Zuerst die **Onboard-LED**, keine Verkabelung nötig:

```
from machine import Pin
import time

led = Pin("LED", Pin.OUT)

for i in range(10):
    led.value(not led.value())   # umschalten
    time.sleep(0.5)
```

Danach eine **externe LED**: GP15 → 220 Ω → langes LED-Bein, kurzes Bein → GND. Im Code `Pin("LED", …)` durch `Pin(15, Pin.OUT)` ersetzen.

ESP32

Zuerst die **Onboard-LED** (bei den meisten DevKit-Boards an GPIO 2):

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)

for i in range(10):
    led.value(not led.value())   # umschalten
    time.sleep(0.5)
```

Keine Onboard-LED oder sie bleibt dunkel? Externe LED: GPIO 18 → 220 Ω → langes LED-Bein, kurzes Bein → GND, im Code `Pin(18, Pin.OUT)`.

Arduino

Nano ESP32: Code wie im ESP32-Tab, Pinnummern laut Pinout deines Boards auf MicroPython-Boards anpassen. **UNO R3 / R4** (kein MicroPython): in Wokwi ein ESP32-Projekt anlegen und den ESP32-Code verwenden.

Ändere es: Lass die LED dreimal kurz und dreimal lang blinken.

4c · Taste schaltet LED (15 min)

Jetzt kommt `umschalten` aus Teil 2 auf das Board, **unverändert**.

micro:bit

```
from microbit import *


def umschalten(zustand):
    if zustand == "AUS":
        return "AN"
    return "AUS"


def anzeigen(zustand):
    if zustand == "AN":
        display.show(Image.HEART)
    else:
        display.clear()


zustand = "AUS"
anzeigen(zustand)

while True:
    if button_a.was_pressed():   # True nur einmal pro Druck
        zustand = umschalten(zustand)
        anzeigen(zustand)
    sleep(20)
```

Raspberry Pi Pico

Aufbau: Taste zwischen **GP14** und **GND**. Externe LED an GP15 wie in 4b (oder Onboard-LED `Pin("LED", Pin.OUT)`).

```
from machine import Pin
import time

led = Pin(15, Pin.OUT)
taste = Pin(14, Pin.IN, Pin.PULL_UP)   # nicht gedrückt = 1, gedrückt = 0
vorher = 1


def gedrueckt():
    """True nur in dem Moment, in dem die Taste hinuntergeht."""
    global vorher
    jetzt = taste.value()
    neu = vorher == 1 and jetzt == 0
    vorher = jetzt
    return neu


def umschalten(zustand):
    if zustand == "AUS":
        return "AN"
    return "AUS"


def anzeigen(zustand):
    led.value(1 if zustand == "AN" else 0)


zustand = "AUS"
anzeigen(zustand)

while True:
    if gedrueckt():
        zustand = umschalten(zustand)
        anzeigen(zustand)
    time.sleep_ms(20)                  # entprellt nebenbei
```

ESP32

Aufbau: Taste zwischen **GPIO 4** und **GND**. LED: Onboard an GPIO 2 oder extern an GPIO 18.

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)
taste = Pin(4, Pin.IN, Pin.PULL_UP)    # nicht gedrückt = 1, gedrückt = 0
vorher = 1
```

```
def gedrueckt():
    """True nur in dem Moment, in dem die Taste hinuntergeht."""
    global vorher
    jetzt = taste.value()
    neu = vorher == 1 and jetzt == 0
    vorher = jetzt
    return neu


def umschalten(zustand):
    if zustand == "AUS":
        return "AN"
    return "AUS"


def anzeigen(zustand):
    led.value(1 if zustand == "AN" else 0)


zustand = "AUS"
anzeigen(zustand)

while True:
    if gedrueckt():
        zustand = umschalten(zustand)
        anzeigen(zustand)
    time.sleep_ms(20)                  # entprellt nebenbei
```

Arduino

Code wie im ESP32-Tab, Pins anpassen (siehe MicroPython-Boards). UNO R3 / R4: Wokwi mit ESP32.

▾ Warum `gedrueckt()` und nicht einfach `taste.value() == 0`?

Die Schleife läuft 50-mal pro Sekunde. Ein Tastendruck dauert etwa 0,2 s, also würde `taste.value() == 0` zehnmal hintereinander zutreffen und die LED zehnmal umschalten. `gedrueckt()` merkt sich in `vorher` den letzten Wert und meldet nur den **Wechsel** von „oben" (1) auf „unten" (0). Das nennt man **Flankenerkennung**. `global vorher` erlaubt der Funktion, die Variable außerhalb zu verändern. Beim micro:bit erledigt das `was_pressed()` für dich.

Ändere es: Übernimm deinen dritten Zustand `BLINKEN` aus Teil 2. Tipp: Im Zustand BLINKEN schaltest du die LED in jedem Schleifendurchlauf um, wenn seit dem letzten Umschalten 300 ms vergangen sind (`time.ticks_ms()`, beim micro:bit `running_time()`).

4d · Ohne PC starten (5 min)

Ein Gadget soll auch ohne Laptop laufen. Speichere das Programm in Thonny mit **Datei → Speichern unter → MicroPython-Gerät** als `main.py`. Nach dem nächsten Einstecken (auch an einer Powerbank) startet es automatisch. Beim micro:bit: **Flash/Send to micro:bit** im Editor.

Videos zum Nachschauen (Pico: externe LED und Taste auslesen, ESP32: erster Einstieg mit Thonny):

▶ *Video: https://www.youtube.com/watch?v=DaZF-3jM69U*

▶ *Video: https://www.youtube.com/watch?v=HTIlg6IqNzA*

▶ *Video: https://www.youtube.com/watch?v=59S53NSHR0M*

5. Ins Team-Repository (5 min)

Lade euren Board-Code ins Team-Repository hoch (**Add file → Upload files**), z. B. als `code/main.py`. Kanban-Board aktualisieren: Was ist **fertig**?

6. Journalauftrag 3 (10 min)

Journalauftrag 3 · Erstes Lebenszeichen

```
# P1 · Woche 3 · <Datum>

## Aufbau
```

```
Foto von eurem Board mit LED und Taste (oder Screenshot aus dem Simulator).
Welche Pins habt ihr verwendet?

## Funktion erklärt
Kopiere eine Funktion aus eurem Code hierher (als ```python-Block)
und erkläre in 2–3 Sätzen: Welche Parameter bekommt sie,
was gibt sie zurück, warum ist sie eine eigene Funktion?

## Problem und Lösung
Was hat nicht auf Anhieb funktioniert, und wie habt ihr es gelöst?
```

Commit-Nachricht: `P1 Woche 3: Board läuft`.

Quiz

Quiz · Woche 3

1. Was gibt dieses Programm aus?

```python
def doppelt(x):
    return 2 * x

print(doppelt(3) + 1)
```

a) `8`

b) `7`

c) `2 * 3 + 1`

d) Nichts, weil `doppelt` nichts ausgibt.

2. `farben = ["rot", "grün", "blau"]`. Was ist `farben[1]`?

a) `"rot"`

b) `"blau"`

c) `"grün"`

d) ein Fehler

3. `i = 3` und die Liste hat 4 Elemente. Was ergibt `(i + 1) % 4`?

a) 0

b) 4

c) 1

d) 3

4. Eine Taste hängt zwischen Pin und GND, der Pin ist mit `Pin.PULL_UP` konfiguriert. Welchen Wert liefert `taste.value()`, während die Taste gedrückt ist?

a) 1

b) `True`

c) Das hängt von der Taste ab.

d) 0

5. Warum trennen wir `umschalten(zustand)` von `anzeigen(zustand)`?

a) Weil Python das so verlangt.

b) Damit die Logik auf PC und Board gleich bleibt und ohne Hardware getestet werden kann.

c) Damit das Programm schneller läuft.

d) Weil eine Funktion höchstens drei Zeilen haben darf.

Bis nächste Woche

- Journalauftrag 3 committen.
- Im Team: Welcher **Sensor** und welcher **Aktor** (Summer, Servo, Display, ...) gehört zu eurem Gadget? Bauteil bei der Lehrkraft reservieren.
- Nächste Woche ist das **Sprint-Review 1**: Euer Board soll dann mindestens eine Muss-Story vorführen können.

WOCHE 4 · SENSOREN, ZUSTANDS-DIAGRAMM, SPRINT-REVIEW

Heute: Euer Gadget bekommt Sinne (Sensor) und eine Stimme (Aktor). Ihr zeichnet, in welchen **Zuständen** es sich befinden kann, und zeigt euren Stand im **Sprint-Review 1**.

Was du am Ende der Stunde hast

- Euer Board hat eine Datei `hardware.py`: Alles, was vom Board abhängt, steckt darin.
- Du hast Sensorwerte live im Plotter gesehen und einen Schwellwert bestimmt.
- Euer Team hat ein **Zustandsdiagramm** und einen **Ablaufplan** des Gadgets.
- Ihr habt beim Sprint-Review vorgeführt, Feedback bekommen und den Backlog für Sprint 2 angepasst.

1. Stand-up (5 min)

Wie immer vor dem Board. Zusätzlich: **Was zeigen wir heute beim Review?** Eine Person bereitet die Vorführung vor.

2. Die Hardware-Schicht · `hardware.py` (15 min)

Letzte Woche stand in jedem Programm board-spezifischer Code (`Pin(14, …)`, `display.show(…)`). Ab heute trennt ihr das sauber:

- `hardware.py` enthält kleine Funktionen wie `helligkeit()`, `led(an)`, `ton(…)`, `gedrueckt()`. Nur diese Datei ist je Board verschieden.
- `main.py` und alle weiteren Programme verwenden nur diese Funktionen. Sie sind für **alle Boards gleich**.

Speichere die passende Datei in Thonny als `hardware.py` **auf das Board** (Datei → Speichern unter → MicroPython-Gerät). Ihr braucht nur die Funktionen, die euer Gadget verwendet. Einen anderen Sensor (Abstand, Bewegung, Temperatur) baut ihr genauso ein: eine Funktion, die eine Zahl liefert. Anschlusspläne für weitere Bauteile: MicroPython-Boards.

micro:bit

Aufbau: nichts, alles eingebaut (micro:bit **V2** für den Ton). Die LED-Anzeige misst gleichzeitig die Helligkeit. Leuchtet sie gerade, stört das die Messung ein wenig.

```
# hardware.py — micro:bit V2
from microbit import *
import music


def helligkeit():
    """Umgebungslicht in Prozent (0 = dunkel, 100 = hell)."""
    return display.read_light_level() * 100 // 255


def neigung():
    """Neigung nach links/rechts, ungefähr -100 bis 100."""
    return accelerometer.get_x() // 10


def led(an):
    if an:
        display.show(Image.HEART)
    else:
        display.clear()


def ton(frequenz, dauer_ms):
    music.pitch(frequenz, dauer_ms)


def gedrueckt():
```

```
    """True einmal pro Druck auf Taste A."""
    return button_a.was_pressed()
```

Raspberry Pi Pico

Aufbau (USB abgezogen):

Bauteil	Anschluss
Taste	GP14 ↔ GND
LED + 220 Ω	GP15 → Widerstand → LED (langes Bein) → GND
Passiver Summer (Piezo)	GP16 ↔ GND
Lichtsensor (LDR) + 10 kΩ	3V3 → LDR → **GP26** → 10 kΩ → GND

Statt des Lichtsensors geht auch ein **Potentiometer** (Außenbeine an 3V3 und GND, Mitte an GP26).

```
# hardware.py — Raspberry Pi Pico
from machine import Pin, ADC, PWM
import time

_taste = Pin(14, Pin.IN, Pin.PULL_UP)
_led = Pin(15, Pin.OUT)
_summer = PWM(Pin(16))
_summer.freq(1000)
_summer.duty_u16(0)
_sensor = ADC(26)
_vorher = 1


def helligkeit():
    """Umgebungslicht in Prozent (0 = dunkel, 100 = hell)."""
    return _sensor.read_u16() * 100 // 65535


def led(an):
    _led.value(1 if an else 0)


def ton(frequenz, dauer_ms):
    _summer.freq(frequenz)
    _summer.duty_u16(32768)            # halbe Periode an = lautester Ton
    time.sleep_ms(dauer_ms)
    _summer.duty_u16(0)


def gedrueckt():
    """True einmal pro Druck (Flankenerkennung)."""
    global _vorher
    jetzt = _taste.value()
    neu = _vorher == 1 and jetzt == 0
    _vorher = jetzt
    return neu
```

ESP32

Aufbau (USB abgezogen):

Bauteil	Anschluss
Taste	GPIO 4 ↔ GND
LED + 220 Ω	GPIO 18 → Widerstand → LED (langes Bein) → GND (oder Onboard-LED GPIO 2)
Passiver Summer (Piezo)	GPIO 19 ↔ GND
Lichtsensor (LDR) + 10 kΩ	3V3 → LDR → **GPIO 34** → 10 kΩ → GND

Statt des Lichtsensors geht auch ein **Potentiometer** (Außenbeine an 3V3 und GND, Mitte an GPIO 34).

```
# hardware.py — ESP32
from machine import Pin, ADC, PWM
import time

_taste = Pin(4, Pin.IN, Pin.PULL_UP)
```

```
_led = Pin(18, Pin.OUT)
_summer = PWM(Pin(19))
_summer.freq(1000)
_summer.duty_u16(0)
_sensor = ADC(Pin(34))
_sensor.atten(ADC.ATTN_11DB)           # voller Bereich 0–3,3 V
_vorher = 1


def helligkeit():
    """Umgebungslicht in Prozent (0 = dunkel, 100 = hell)."""
    return _sensor.read_u16() * 100 // 65535


def led(an):
    _led.value(1 if an else 0)


def ton(frequenz, dauer_ms):
    _summer.freq(frequenz)
    _summer.duty_u16(32768)
    time.sleep_ms(dauer_ms)
    _summer.duty_u16(0)


def gedrueckt():
    """True einmal pro Druck (Flankenerkennung)."""
    global _vorher
    jetzt = _taste.value()
    neu = _vorher == 1 and jetzt == 0
    _vorher = jetzt
    return neu
```

Arduino

Nano ESP32: ESP32-Datei verwenden, Pinnummern laut Pinout auf MicroPython-Boards anpassen. Die Funktionsnamen bleiben gleich, dann läuft der restliche Code unverändert. **UNO R3 / R4** (kein MicroPython): Wokwi mit ESP32 und derselben Verkabelung.

Der Unterstrich vor `_taste`, `_led` … bedeutet: nur für den Gebrauch **innerhalb** von `hardware.py`.

3. Sensor erkunden · `sensor_test.py` (10 min)

Dieses Programm ist für alle Boards gleich:

```
from hardware import *
import time

while True:
    print(helligkeit())
    time.sleep_ms(200)
```

Starte es und öffne in Thonny **Ansicht → Plotter**. Deck den Sensor mit der Hand ab, leuchte mit dem Handy darauf.

Bestimme im Team: Welcher Wert ist „hell" (Tageslicht im Raum), welcher „dunkel" (Hand drüber)? Wählt einen **Schwellwert** dazwischen und notiert ihn.

4. Sensor steuert Aktor · `nachtlicht_einfach.py` (10 min)

```
from hardware import *
import time

SCHWELLE = 30          # euer Wert aus Teil 3


def ist_dunkel(wert):
    return wert < SCHWELLE


while True:
    led(ist_dunkel(helligkeit()))
    time.sleep_ms(100)
```

Probier aus und beobachte:

1. Halte die Hand langsam über den Sensor, genau an der Grenze. Was macht die LED?
2. Wie würdest du das Nachtlicht **ganz ausschalten**, wenn du es nicht brauchst?

Frage 2 kann dieses Programm nicht lösen: Es kennt keine **Zustände**. Es macht immer dasselbe, egal was vorher war. Dafür braucht ihr ein Modell.

Was passiert bei 1.?

Die LED flackert. Der Messwert schwankt um den Schwellwert herum, jeder kleine Ausschlag schaltet um. Nächste Woche untersucht ihr, wie man das verhindert.

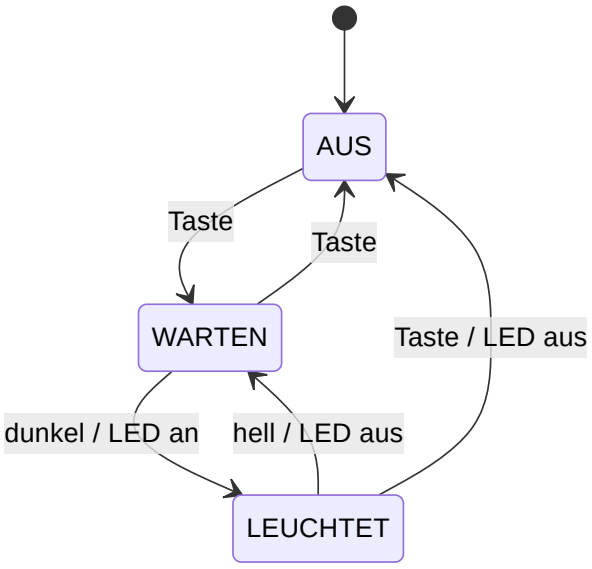
5. Zustandsdiagramm und Ablaufplan (15 min)

5a · Zustandsdiagramm: *In welchem Zustand ist das Gerät?*

Ein **Zustandsdiagramm** zeigt

- **Zustände** (abgerundete Kästen): Situationen, in denen sich das Gerät gleich verhält,
- **Übergänge** (Pfeile): *Ereignis / Aktion*, z. B. `Taste / LED aus`,
- den **Startzustand** (Pfeil vom schwarzen Punkt).

Das Nachtlicht mit Aus-Taste:

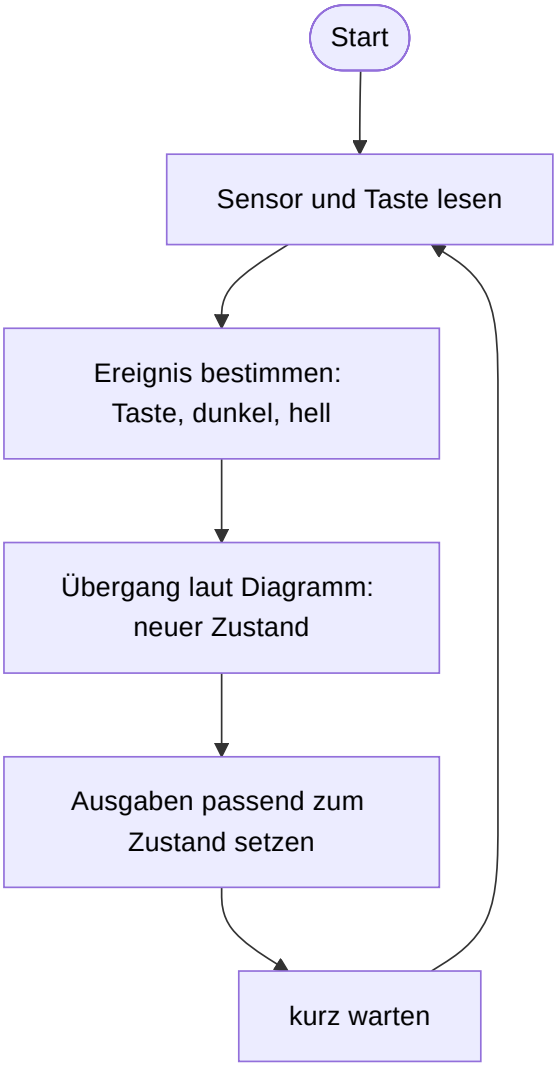


Prüfe das Diagramm: Gibt es für jeden Zustand eine Antwort auf jedes Ereignis? Was passiert im Zustand AUS, wenn es dunkel wird? (Nichts. Ein fehlender Pfeil heißt: Der Zustand bleibt.)

▶ *Video: https://www.youtube.com/watch?v=nwmIg0ZYnF8*

5b · Ablaufplan: *Was tut das Programm, Schritt für Schritt?*

Das Zustandsdiagramm sagt, **was** das Gerät tut. Der **Ablaufplan** zeigt, **wie** die Hauptschleife das umsetzt. Beide braucht ihr: Das eine modelliert das Gerät, das andere das Programm.



Auftrag im Team: Zeichnet das Zustandsdiagramm **eures** Gadgets, zuerst auf Papier. Legt es im Team-Repository als `ZUSTANDSDIAGRAMM.md` ab, entweder als Foto oder als Mermaid-Block (GitHub zeigt ihn als Grafik an):

````
```mermaid
stateDiagram-v2
    [*] --> AUS
    AUS --> WARTEN: Taste
```
````

Checkliste Zustandsdiagramm

- Jeder Zustand hat einen Namen, der eine **Situation** beschreibt (WARTEN, LEUCHTET), keine Tätigkeit des Programms (SENSOR_LESEN).
- Genau ein Startzustand.
- Jeder Pfeil ist mit einem **Ereignis** beschriftet.

- Von jedem Zustand kommt man wieder weg (keine Sackgasse, außer das ist gewollt).
- Eine Person, die euer Gerät nicht kennt, kann anhand des Diagramms vorhersagen, was ein Tastendruck bewirkt.

6. Sprint-Review 1 (35 min)

Das **Sprint-Review** ist eine Zwischenpräsentation: Ihr zeigt, was **läuft**, nicht was ihr vorhabt. Das Publikum sind eure Mitschüler:innen, sie geben Feedback wie Kund:innen.

Pro Team 4 Minuten:

Zeit	Inhalt
2 min	**Demo** am Board: welche User Stories sind fertig? Akzeptanzkriterien laut vorlesen und vorführen.
1 min	**Zustandsdiagramm** zeigen: So soll das fertige Gadget funktionieren.
1 min	**Feedback** aus der Klasse

Feedback geben (kurz, freundlich, konkret):

- *Mir gefällt …*
- *Ich frage mich …*
- *Wie wäre es, wenn …*

Nach allen Vorführungen (5 min im Team): Feedback auf neue Karten schreiben, Backlog neu sortieren, Karten für **Sprint 2** (Woche 5–6) auswählen. Unfertiges aus Sprint 1 wandert zurück in den Backlog, nicht automatisch in Sprint 2.

7. Journalauftrag 4 (5 min, Rest zu Hause)

Journalauftrag 4 · Modell und Review

```
# P1 · Woche 4 · <Datum>

## Unser Modell
Zustandsdiagramm eures Gadgets (Foto oder Mermaid-Block).
Erkläre einen Übergang in eigenen Worten:
„Im Zustand … führt … dazu, dass …"

## Sensor
Welcher Sensor, welcher Schwellwert, wie habt ihr ihn gefunden?

## Feedback aus dem Review
Das wichtigste Feedback, das ihr bekommen habt,
und was ihr in Sprint 2 deshalb ändert.
```

Commit-Nachricht: `P1 Woche 4: Zustandsdiagramm und Review`.

Quiz

Quiz · Woche 4

1. Warum steckt der board-spezifische Code in `hardware.py`?

a) Weil MicroPython nur zwei Dateien erlaubt.

b) Damit der restliche Code auf allen Boards gleich bleibt und man das Board wechseln kann, ohne alles umzuschreiben.

c) Damit das Programm schneller startet.

d) Weil `main.py` keine Imports enthalten darf.

2. Welcher dieser Namen ist ein guter **Zustand** für ein Zustandsdiagramm?

a) `SENSOR_LESEN`

b) `WHILE_SCHLEIFE`

c) `ALARM_AKTIV`

d) `LED_AN_SCHALTEN`

3. Im Diagramm oben: Das Nachtlicht ist im Zustand **AUS** und es wird dunkel. Was passiert?

a) Nichts, es bleibt im Zustand AUS.

b) Es wechselt nach LEUCHTET.

c) Es wechselt nach WARTEN.

d) Das Diagramm ist fehlerhaft.

4. Was ist der Unterschied zwischen Zustandsdiagramm und Ablaufplan?

a) Es gibt keinen, das sind zwei Namen für dasselbe.

b) Der Ablaufplan zeigt die Zustände, das Zustandsdiagramm den Code.

c) Das Zustandsdiagramm ist für Python, der Ablaufplan für MicroPython.

d) Das Zustandsdiagramm modelliert, wie sich das Gerät verhält. Der Ablaufplan zeigt, in welcher Reihenfolge das Programm arb

5. Was gehört beim Sprint-Review in die Vorführung?

a) Alles, was ihr bis zum Projektende noch bauen wollt.

b) Was jetzt schon funktioniert, gemessen an den Akzeptanzkriterien.

c) Der gesamte Quellcode, Zeile für Zeile.

d) Nur Folien, damit nichts schiefgehen kann.

Bis nächste Woche

- Journalauftrag 4 committen.
- `ZUSTANDSDIAGRAMM.md` im Team-Repository fertigstellen: Nächste Woche wird es zu Code.
- Wer mag, zum Anschauen: Wie reagiert ein Sensor, und wie programmiert man ihn (am micro:bit gezeigt, gilt für alle Boards)?

▶ *Video: https://www.youtube.com/watch?v=rqTMaD4FnFA*

WOCHE 5 · ZUSTANDSAUTOMAT UND EXPERIMENT

Heute: Sprint 2 beginnt. Du übersetzt ein Zustandsdiagramm nach einem festen Rezept in Code, baust ein Reaktionsspiel und untersuchst im **Experiment**, wie sich das Verhalten eines interaktiven Systems ändert, wenn du Eingaben und Rückmeldungen variierst.

Was du am Ende der Stunde hast

- Du kannst ein Zustandsdiagramm mit dem **Automaten-Rezept** in Python umsetzen.
- Das Nachtlicht mit Aus-Taste läuft (am PC simuliert und/oder am Board).
- Du hast Reaktionszeiten unter verschiedenen Bedingungen gemessen und das Ergebnis erklärt.
- Euer eigenes Gadget hat einen Zustandsautomaten nach dem Rezept, mindestens für einen Teil der Zustände.

1. Stand-up (5 min)

Sprint 2 hat begonnen: Welche Karten habt ihr nach dem Review ausgewählt? Wer nimmt welche?

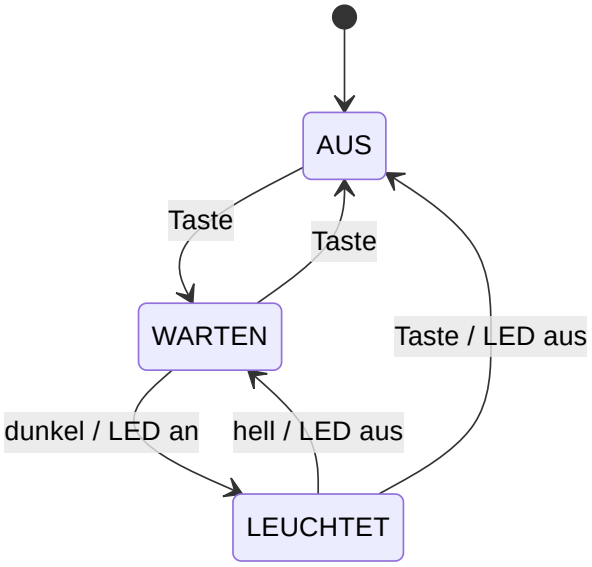
2. Vom Diagramm zum Code (30 min)

2a · Das Automaten-Rezept

Jedes Gadget mit Zuständen kann dieselbe Struktur haben:

1. **Eine** Variable `zustand` hält den aktuellen Zustand.
2. Eine Funktion `naechster_zustand(zustand, ereignis)` enthält **genau die Pfeile** des Diagramms: ein `if`-Block pro Zustand, darin ein `if` pro ausgehendem Pfeil. Kein Pfeil passt? Zustand bleibt.
3. Die Hauptschleife: **Ereignis bestimmen → neuen Zustand berechnen → bei einem Wechsel die Ausgaben setzen**.

Zur Erinnerung, das Diagramm aus Woche 4:



2b · Am PC simulieren · `nachtlicht_pc.py`

Die Ereignisse tippst du ein, statt sie zu messen. So testest du die Logik ohne Hardware.

```
def naechster_zustand(zustand, ereignis):
    """Die Pfeile des Zustandsdiagramms, sonst nichts."""
    if zustand == "AUS":
        if ereignis == "taste":
            return "WARTEN"
    elif zustand == "WARTEN":
        if ereignis == "taste":
            return "AUS"
        if ereignis == "dunkel":
            return "LEUCHTET"
    elif zustand == "LEUCHTET":
        if ereignis == "taste":
            return "AUS"
        if ereignis == "hell":
            return "WARTEN"
    return zustand                      # kein passender Pfeil: bleibt


zustand = "AUS"
print("Ereignisse: taste, dunkel, hell · q beendet")

while True:
    ereignis = input(zustand + " > ")
    if ereignis == "q":
        break
    neu = naechster_zustand(zustand, ereignis)
    if neu != zustand:
        print("  ", zustand, "→", neu, "| LED", "an" if neu == "LEUCHTET" else "aus")
        zustand = neu
```

Probier aus: Spiel das Diagramm Pfeil für Pfeil durch. Tippe auch Ereignisse, die in einem Zustand **keinen** Pfeil haben (z. B. `dunkel` im Zustand AUS).

2c · Auf dem Board · `main.py`

`naechster_zustand` wird **Zeichen für Zeichen** übernommen. Nur woher die Ereignisse kommen und was bei einem Wechsel passiert, ändert sich. Das Programm verwendet eure `hardware.py` aus Woche 4 und ist damit für alle Boards gleich.

```
from hardware import *
import time

SCHWELLE = 30


def naechster_zustand(zustand, ereignis):
    if zustand == "AUS":
        if ereignis == "taste":
            return "WARTEN"
    elif zustand == "WARTEN":
        if ereignis == "taste":
            return "AUS"
        if ereignis == "dunkel":
            return "LEUCHTET"
    elif zustand == "LEUCHTET":
        if ereignis == "taste":
            return "AUS"
        if ereignis == "hell":
            return "WARTEN"
    return zustand


def ereignis_bestimmen():
    if gedrueckt():
        return "taste"
    if helligkeit() < SCHWELLE:
        return "dunkel"
    return "hell"


zustand = "AUS"
led(False)

while True:
    neu = naechster_zustand(zustand, ereignis_bestimmen())
    if neu != zustand:
        print(zustand, "→", neu)
        zustand = neu
        led(zustand == "LEUCHTET")
```

```
    time.sleep_ms(50)
```

Ändere es · das Flackern beheben: In Woche 4 hat die LED an der Grenze geflackert. Verwende **zwei** Schwellwerte: „dunkel" erst unter 25, „hell" erst über 35. Dazwischen gibt es **kein** Ereignis, der Zustand bleibt. (Fachwort: **Hysterese**, so arbeitet auch ein Thermostat.)

Lösung

```
SCHWELLE_DUNKEL = 25
SCHWELLE_HELL = 35


def ereignis_bestimmen():
    if gedrueckt():
        return "taste"
    wert = helligkeit()
    if wert < SCHWELLE_DUNKEL:
        return "dunkel"
    if wert > SCHWELLE_HELL:
        return "hell"
    return None                          # Graubereich: kein Ereignis
```

`naechster_zustand` bleibt unverändert, weil `None` zu keinem Pfeil passt.

Profi-Variante: Übergänge als Tabelle

Die Pfeile lassen sich auch als **Dictionary** speichern. Der Schlüssel ist das Paar (Zustand, Ereignis), der Wert der neue Zustand. Dann ist der Code fast wörtlich das Diagramm:

```
UEBERGAENGE = {
    ("AUS", "taste"): "WARTEN",
    ("WARTEN", "taste"): "AUS",
    ("WARTEN", "dunkel"): "LEUCHTET",
    ("LEUCHTET", "taste"): "AUS",
    ("LEUCHTET", "hell"): "WARTEN",
}

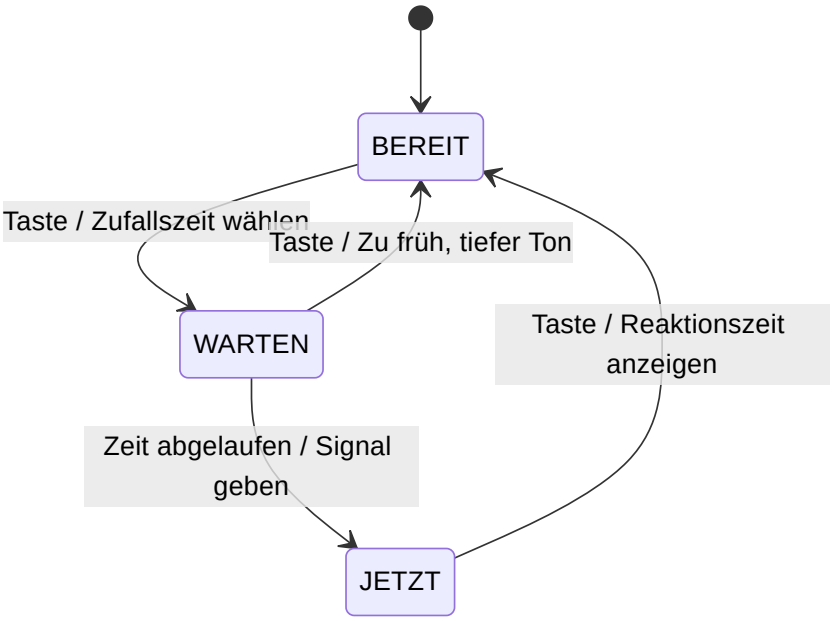

def naechster_zustand(zustand, ereignis):
    return UEBERGAENGE.get((zustand, ereignis), zustand)
```

Mehr zu Dictionaries: Selbstlernen: Datenstrukturen.

▶ *Video: https://www.youtube.com/watch?v=3NroZHXFihE*

3. Reaktionsspiel · `reaktion.py` (15 min)

Ein Automat, bei dem **Zeit** ein Ereignis ist. Auch dieses Programm ist mit eurer `hardware.py` für alle Boards gleich. Ohne Lichtsensor im Aufbau? Die Funktion `helligkeit` wird hier nicht gebraucht.




```python
from hardware import *
import time
import random

# Versuchsbedingungen für das Experiment
SIGNAL_LED = True
SIGNAL_TON = True
ZUFALLS_WARTEZEIT = True

zustand = "BEREIT"
startzeit = 0
wartezeit = 0
zeiten = []
print("Taste drücken zum Starten")

while True:
    taste = gedrueckt()
    jetzt = time.ticks_ms()

    if zustand == "BEREIT":
        if taste:
            wartezeit = random.randint(1000, 4000) if ZUFALLS_WARTEZEIT else 2000
            startzeit = jetzt
            zustand = "WARTEN"

    elif zustand == "WARTEN":
        if taste:
            print("Zu früh!")
            ton(220, 300)
            zustand = "BEREIT"
        elif time.ticks_diff(jetzt, startzeit) >= wartezeit:
            startzeit = time.ticks_ms()
            if SIGNAL_LED:
                led(True)
            if SIGNAL_TON:
                ton(880, 100)
            zustand = "JETZT"

    elif zustand == "JETZT":
        if taste:
            led(False)
            reaktion = time.ticks_diff(jetzt, startzeit)
            zeiten.append(reaktion)
            print("Versuch " + str(len(zeiten)) + ":", reaktion, "ms | Mittelwert:",
                  sum(zeiten) // len(zeiten), "ms")
            zustand = "BEREIT"

    time.sleep_ms(5)
```

`time.ticks_ms()` zählt Millisekunden seit dem Start des Boards, `time.ticks_diff(a, b)` rechnet sicher `a - b` aus.

Spielt ein paar Runden. Vergleicht das Programm mit dem Diagramm: Wo steckt jeder Pfeil im Code?

4. Experiment: Systemverhalten untersuchen (25 min)

Ein interaktives System besteht aus **Eingabe** (Taste), **Verarbeitung** (Automat) und **Rückmeldung** (LED, Ton). Wie schnell und wie zuverlässig ein Mensch damit umgeht, hängt stark von der Rückmeldung ab. Das messt ihr jetzt.

1. **Vermutung aufschreiben** (bevor ihr messt): Bei welcher Bedingung reagiert ihr am schnellsten? Warum?
2. **Messen:** Für jede Bedingung die Konstanten oben im Programm ändern, neu starten, **5 gültige Versuche** pro Person. Mittelwert und Anzahl „Zu früh!" notieren.

#	`SIGNAL_LED`	`SIGNAL_TON`	`ZUFALLS_WARTEZEIT`	Mittelwert (ms)	„Zu früh!"
A	True	False	True		
B	False	True	True		
C	True	True	True		
D	True	True	**False**		

3. **Erklären:** Stimmt eure Vermutung? Woran könnte der Unterschied zwischen A und B liegen? Was passiert bei D mit der Reaktionszeit, und was mit „Zu früh!"? Warum?
4. **Weiterdenken:** Welche Rückmeldung wäre für eine Person, die schlecht sieht, sinnvoll? Für eine, die schlecht hört?

Hintergrund, erst nach dem Messen aufklappen

Viele Menschen reagieren auf einen **Ton** etwas schneller als auf ein **Lichtsignal**, weil das Gehirn akustische Reize schneller verarbeitet. Bei einer **festen Wartezeit** (D) lernt man den Rhythmus und drückt **vorausschauend**: Die gemessene Zeit wird kürzer, aber es gibt mehr Fehlstarts. Das Gerät misst dann nicht mehr die Reaktion, sondern das Raten. Eure Werte können anders aussehen: Ein paar Versuche sind eine kleine Stichprobe, und ein blockierender `ton()` oder die Schleifenpause verfälschen um einige Millisekunden. Genau solche Einflüsse gehören in die Erklärung.

5. Auf euer Gadget übertragen (10 min)

Setzt euer eigenes Zustandsdiagramm aus Woche 4 mit dem Rezept um: zuerst `naechster_zustand` (am PC mit eingetippten Ereignissen testen), dann `ereignis_bestimmen` mit euren Sensoren. Beantwortet im Team:

- Welche **Rückmeldung** bekommt die Person, die euer Gadget benutzt, bei jedem Übergang?
- Was passiert bei einer **unerwarteten Eingabe** (Taste im falschen Moment, Sensor verdeckt)?

Neue Erkenntnisse als Karten auf das Kanban-Board.

6. Journalauftrag 5 (10 min)

Journalauftrag 5 · Messprotokoll

```
# P1 · Woche 5 · <Datum>

## Vermutung
Was hast du vor dem Experiment erwartet?

## Messwerte
Die Tabelle A–D mit deinen Werten.

## Erklärung
Stimmt die Vermutung? Wie erklärst du die Unterschiede?
Was bedeutet das für die Rückmeldungen eures eigenen Gadgets?

## Automat
Ein Ausschnitt aus `naechster_zustand` eures Gadgets
und der passende Teil des Zustandsdiagramms.
```

Commit-Nachricht: `P1 Woche 5: Automat und Experiment`.

Quiz

Quiz · Woche 5

1. Was gehört laut Automaten-Rezept in die Funktion `naechster_zustand`?

a) Alle Ausgaben wie `led()` und `ton()`.

b) Nur die Übergänge (Pfeile) des Zustandsdiagramms.

c) Das Lesen der Sensoren.

d) Die Hauptschleife.

2. Was liefert `naechster_zustand("LEUCHTET", "dunkel")` im Nachtlicht?

a) `"AUS"`

b) `"WARTEN"`

c) `"LEUCHTET"`

d) `None`

3. Warum hilft eine Hysterese (zwei Schwellwerte) gegen Flackern?

a) Im Bereich zwischen den Schwellen gibt es kein Ereignis, kleine Schwankungen lösen also keinen Wechsel mehr aus.

b) Der Sensor misst dadurch genauer.

c) Die Schleife läuft langsamer.

d) Die LED bekommt weniger Strom.

4. Bei fester Wartezeit (Bedingung D) wird die gemessene Reaktionszeit oft kürzer. Was ist die beste Erklärung?

a) Das Board rechnet schneller.

b) Der Ton ist lauter.

c) Die Personen sind nach mehreren Versuchen einfach wacher.

d) Man lernt den Rhythmus und drückt vorausschauend, statt auf das Signal zu reagieren.

Bis nächste Woche

- Journalauftrag 5 committen.
- `naechster_zustand` eures Gadgets fertig und am PC durchgespielt.
- Nächste Woche: Testen, Code-Review, README und Präsentation vorbereiten. Die Präsentation ist in **zwei Wochen**.

WOCHE 6 · TESTEN, CODE-REVIEW, PRÄSENTATION

Heute: Ihr macht das Gadget fertig und belastbar: Logik automatisch testen, Gerät nach Protokoll prüfen, Code gegenseitig begutachten, README schreiben und die Präsentation für nächste Woche vorbereiten.

Was du am Ende der Stunde hast

- Die Logik eures Gadgets steckt in `logik.py` und wird von `test_logik.py` am PC geprüft.
- `TESTS.md` enthält ein Testprotokoll des echten Geräts.
- Ein anderes Team hat euren Code begutachtet, ihr habt die wichtigsten Punkte verbessert.
- `README.md` im Team-Repository beschreibt das Gadget vollständig.
- Die 3-Minuten-Präsentation ist geplant und einmal geprobt.

1. Stand-up (5 min)

Zusätzlich heute: **Was muss bis zum Review unbedingt noch fertig werden, was streichen wir?** Karten, die nicht mehr realistisch sind, zurück in den Backlog. Ein kleines Gadget, das zuverlässig läuft, schlägt ein großes, das bei der Vorführung abstürzt.

2. Logik testen ohne Hardware (20 min)

2a · Drei Dateien

Euer Projekt bekommt seine endgültige Struktur:

Datei	Inhalt	läuft auf
`hardware.py`	Pins, Sensoren, Aktoren (Woche 4)	nur Board
`logik.py`	`naechster_zustand` und andere Funktionen **ohne** Hardware	Board **und** PC
`main.py`	Hauptschleife: Ereignis bestimmen → `naechster_zustand` → Ausgaben	nur Board

`logik.py` für das Nachtlicht:

```
# logik.py — keine Imports von hardware!

def naechster_zustand(zustand, ereignis):
    if zustand == "AUS":
        if ereignis == "taste":
            return "WARTEN"
    elif zustand == "WARTEN":
        if ereignis == "taste":
            return "AUS"
        if ereignis == "dunkel":
            return "LEUCHTET"
    elif zustand == "LEUCHTET":
        if ereignis == "taste":
            return "AUS"
        if ereignis == "hell":
            return "WARTEN"
    return zustand
```

In `main.py` ersetzt ihr die Funktion durch `from logik import naechster_zustand`. `logik.py` muss auch auf dem Board gespeichert sein.

2b · `test_logik.py`

Jeder Pfeil im Zustandsdiagramm wird ein Testfall, dazu ein paar Fälle **ohne** Pfeil. Speichert `logik.py` und `test_logik.py` im selben Ordner **am PC** und startet den Test in Thonny mit dem Interpreter **Lokales Python 3**.

```
# test_logik.py – läuft am PC
from logik import naechster_zustand

TESTS = [
    # (Zustand,   Ereignis, erwarteter neuer Zustand)
    ("AUS",      "taste",  "WARTEN"),
    ("WARTEN",   "taste",  "AUS"),
    ("WARTEN",   "dunkel", "LEUCHTET"),
    ("LEUCHTET", "hell",   "WARTEN"),
    ("LEUCHTET", "taste",  "AUS"),
    # Ereignisse ohne Pfeil: Zustand bleibt
    ("AUS",      "dunkel", "AUS"),
    ("WARTEN",   "hell",   "WARTEN"),
    ("LEUCHTET", "dunkel", "LEUCHTET"),
]

bestanden = 0
for zustand, ereignis, erwartet in TESTS:
    ergebnis = naechster_zustand(zustand, ereignis)
    if ergebnis == erwartet:
        bestanden = bestanden + 1
        print("ok     ", zustand, "+", ereignis, "→", ergebnis)
    else:
        print("FEHLER ", zustand, "+", ereignis, "→", ergebnis, "erwartet:", erwartet)

print(bestanden, "von", len(TESTS), "Tests bestanden")
```

Probier aus:

1. Alle Tests grün? Dann baue **absichtlich einen Fehler** in `logik.py` ein (z. B. `return "AUS"` statt `return "WARTEN"`). Findet der Test ihn?
2. Schreibt `logik.py` und `test_logik.py` für **euer** Gadget. Ein Testfall pro Pfeil eures Diagramms.

<details>
<summary>Kurzform mit <code>assert</code></summary>

Profis schreiben Tests oft mit `assert`: Die Zeile tut nichts, wenn die Bedingung stimmt, und bricht mit `AssertionError` ab, wenn nicht.

```
assert naechster_zustand("AUS", "taste") == "WARTEN"
assert naechster_zustand("AUS", "dunkel") == "AUS"
print("alle Tests bestanden")
```
</details>

2c · Testprotokoll am Gerät

Nicht alles lässt sich am PC testen: Wackelkontakte, Schwellwerte, Tonlautstärke. Dafür gibt es das **Testprotokoll**, eine Tabelle in `TESTS.md`. Eine Person bedient, eine liest vor und trägt ein. Grundlage sind eure Akzeptanzkriterien aus `ANFORDERUNGEN.md`.

```
| # | Ausgangszustand | Aktion | Erwartet | Beobachtet | ok? |
|---|-----------------|--------|----------|------------|-----|
| 1 | AUS, Raum hell | Taste drücken | WARTEN, LED aus | LED aus | ✅ |
| 2 | WARTEN | Hand über Sensor | LED an innerhalb 1 s | LED an nach ca. 0,5 s | ✅ |
| 3 | LEUCHTET | Taste drücken | AUS, LED aus | LED bleibt an | ❌ |
```

Jeder ❌ wird eine Karte auf dem Kanban-Board.

3. Code-Review im Teamtausch (20 min)

Zwei Teams tauschen die Plätze und lesen gegenseitig den Code (10 min pro Richtung). Die Gutachter:innen schreiben ihre Punkte als **Issue** ins Team-Repository des anderen Teams (Reiter **Issues** → **New issue**) oder auf Zettel.

Checkliste Code-Review

1. **Läuft es?** `main.py` startet ohne Fehler, `test_logik.py` ist grün.
2. **Passt der Code zum Diagramm?** Jeder Pfeil ist in `naechster_zustand` zu finden, und umgekehrt.
3. **Namen:** Sagen Variablen- und Funktionsnamen, was drin ist? (`schwelle_dunkel` statt `x`)
4. **Keine magischen Zahlen:** Pins, Schwellwerte, Zeiten stehen als KONSTANTEN oben oder in `hardware.py`.
5. **Kommentare** erklären das **Warum**, nicht das Was. (`# Hysterese gegen Flackern`, nicht `# x um 1 erhöhen`)
6. **Kurze Funktionen:** Keine Funktion ist länger als eine Bildschirmseite.
7. **Eine Sache**, die euch besonders gut gefällt.

Punkte 1–2 sind Pflicht vor der Präsentation. Den Rest verbessert ihr, soweit Zeit bleibt. Mehr dazu, wie man Code systematisch verbessert: Selbstlernen: Code-Qualität.

4. README: die Visitenkarte des Projekts (15 min)

Die `README.md` ist das Erste, was man im Repository sieht. Sie ist später auch die Grundlage für die Projektseite in eurem Web-Portfolio (P5).

```
# <Name des Gadgets>

<Ein Satz: Was macht es, für wen?>

![Foto des Gadgets](images/gadget.jpg)

## Funktionen
- <die fertigen User Stories in einem Satz je Story>

## Hardware
| Bauteil | Anschluss |
|---------|-----------|
| <Board> | – |
| <Taste> | <Pin> ↔ GND |

## Starten
1. `hardware.py`, `logik.py` und `main.py` auf das Board kopieren.
2. Board einstecken, das Programm startet automatisch.

## Zustandsdiagramm
```mermaid
stateDiagram-v2
    [*] --> ...
```

Tests
`test_logik.py` am PC ausführen. Testprotokoll: TESTS.md

Team
<Namen oder GitHub-Benutzernamen, wer hat was gemacht>

Quellen
- <Links zu Anleitungen und Code, die ihr verwendet habt>
- Fotos: eigene Aufnahmen
```

⚠ Fremdes Material

Code, Bilder oder Schaltpläne aus dem Internet dürft ihr nur verwenden, wenn die Lizenz es erlaubt, und ihr müsst die **Quelle nennen**. Fotos von Personen nur mit deren Zustimmung. Das gilt ab jetzt für jedes Projekt.

5. Präsentation vorbereiten (25 min)

Nächste Woche hat jedes Team **3 Minuten** plus 2 Minuten Fragen. Keine Folienschlacht: Das Gadget ist der Star.

Zeit	Inhalt	Tipp
0:30	**Problem und Idee:** Für wen, wozu?	mit einer Alltagssituation beginnen
1:00	**Live-Demo**	vorher üben, wer was drückt. Die Demo folgt einer User Story.

Zeit	Inhalt	Tipp
0:45	**Zustandsdiagramm** und ein **Code-Ausschnitt**, auf den ihr stolz seid	groß genug für die letzte Reihe
0:30	**Gelernt und schwierig:** ein echter Stolperstein und wie ihr ihn gelöst habt	ehrlich, nicht „alles super"
0:15	**Ausblick:** Was würdet ihr als Nächstes bauen?	

Plan B

Nehmt heute ein **Handyvideo** (30 s) von der funktionierenden Demo auf. Wenn am Präsentationstag ein Kabel streikt, zeigt ihr das Video und erklärt, was schiefging. Das ist keine Schande, sondern Alltag in der Technik.

Einmal proben mit Stoppuhr, ein anderes Team schaut zu und sagt: Was war unklar? War es unter 3 Minuten?

6. Journalauftrag 6 (10 min)

Journalauftrag 6 · Fehler und Rolle

```
# P1 · Woche 6 · <Datum>

## Ein Fehler, den wir gefunden haben
Wie ist er aufgefallen (Test, Testprotokoll, Code-Review, Zufall)?
Was war die Ursache, wie habt ihr ihn behoben?

## Code-Review
Das nützlichste Feedback zu eurem Code, und ein Punkt,
den du selbst beim anderen Team gelernt hast.

## Meine Rolle
Was ist dein Beitrag zum Gadget? Verlinke einen Commit oder eine Datei.
```

Commit-Nachricht: `P1 Woche 6: Tests und Review`.

Quiz

Quiz · Woche 6

1. Warum darf `logik.py` nichts aus `hardware.py` importieren?

a) Weil Python zirkuläre Imports verbietet.

b) Damit die Datei kleiner ist.

c) Damit `logik.py` am PC ohne Board ausgeführt und getestet werden kann.

d) Weil `hardware.py` nur einmal importiert werden darf.

2. Das Zustandsdiagramm hat 5 Pfeile. Wie viele Testfälle braucht `test_logik.py` mindestens?

a) 1

b) 5, einen pro Pfeil, und besser noch ein paar für Ereignisse ohne Pfeil.

c) So viele, wie `logik.py` Zeilen hat.

d) Keinen, wenn das Gerät funktioniert.

3. Welcher Kommentar ist am hilfreichsten?

a) `# Variable schwelle`

b) `# if-Abfrage`

c) `# schwelle um 5 erhöhen`

d) `# zwei Schwellen, damit die LED an der Grenze nicht flackert`

4. Was gehört **nicht** ins Testprotokoll am Gerät?

a) Wie viele Zeilen Code das Programm hat.

b) Der Ausgangszustand.

c) Was erwartet wurde.

d) Was tatsächlich beobachtet wurde.

Bis nächste Woche

- Journalauftrag 6 committen.
- Offene ❌ aus dem Testprotokoll beheben, `README.md` fertigstellen.
- Präsentation zu Hause noch einmal durchgehen, Plan-B-Video ins Team-Repository legen.

WOCHE 7 · REVIEW UND RETROSPEKTIVE

Heute: Das große Finale von P1. Ihr präsentiert eure Gadgets, schaut in der **Retrospektive** darauf, wie ihr zusammengearbeitet habt, und schließt euer Lernjournal für dieses Projekt ab.

Was du am Ende der Stunde hast

- Euer Team hat das Gadget präsentiert und Feedback bekommen.
- Du hast mindestens einem anderen Team schriftliches Feedback gegeben.
- Euer Team hat 1–2 konkrete Vorsätze für P2.
- Dein Journal für P1 ist vollständig, mit Projektrückblick und Selbsteinschätzung, und committet.

1. Aufbauen und Technik-Check (10 min)

- Gadget aufbauen, an Strom (Powerbank oder USB) anschließen, **einmal komplett durchspielen**.
- Zustandsdiagramm und Code-Ausschnitt für die Projektion bereit (Team-Repository im Browser geöffnet).
- Plan-B-Video griffbereit.
- Reihenfolge der Teams steht an der Tafel. Jede Person bekommt **ein Team zugelost**, dem sie schriftlich Feedback gibt.

2. Präsentationen (45 min)

Pro Team **3 Minuten Präsentation + 2 Minuten Fragen**, Ablauf wie in Woche 6 geplant. Die Lehrkraft stoppt die Zeit.

Feedback-Karte (für das zugeloste Team, auf Papier oder als Issue im Team-Repository):

```
Team: …

Mir gefällt: …
Ich frage mich: …
Wie wäre es, wenn: …

Die Demo hat gezeigt, dass … (welche User Story?)
```

Worauf die Lehrkraft achtet (Produkt, Prozess, Präsentation, Journal), steht in den Rubrik-Vorlagen.

3. Retrospektive (20 min)

Das Review schaut auf das **Produkt**. Die **Retrospektive** schaut auf die **Zusammenarbeit**: Wie haben wir gearbeitet, und was machen wir im nächsten Projekt anders? Es geht nicht um Schuld, sondern um Verbesserung.

Format **Start · Stop · Continue**, im Team:

Schritt	Zeit	Was passiert
Still schreiben	5 min	Jede Person schreibt Haftnotizen in drei Spalten: **Start** (Was sollten wir anfangen?), **Stop** (Was sollten wir lassen?), **Continue** (Was hat gut funktioniert?). Eine Idee pro Zettel.
Vorstellen und ordnen	8 min	Reihum vorlesen und aufkleben, Ähnliches zusammenschieben.
Entscheiden	5 min	Jede Person hat 2 Punkte zum Vergeben. Die 1–2 Zettel mit den meisten Punkten werden **Vorsätze für P2**, formuliert als konkrete Handlung: „Wir machen jede Stunde ein Stand-up mit Board-Foto" statt „besser kommunizieren".

Schritt	Zeit	Was passiert
Festhalten	2 min	Foto der Wand, Vorsätze in die `README.md` des Team-Repositorys unter **Retrospektive**.

Denkanstöße, falls euch nichts einfällt: Kanban-Board aktuell gehalten? Aufgaben fair verteilt? Wann habt ihr am meisten geschafft, wann am wenigsten, und warum? Wie lief das Zusammenführen des Codes?

▶ *Video: https://www.youtube.com/watch?v=rWz9VsHSpAE*

4. Journal-Abgabe (15 min)

Abgabe heißt: **der letzte Commit ist drin**. Es wird nichts eingesammelt (siehe Lernjournal).

Journalauftrag 7 · Projektrückblick P1

Datei `p1/woche-07.md`:

```
# P1 · Woche 7 · Projektrückblick · <Datum>

## Unser Gadget
Ein Satz, ein Foto, Link zum Team-Repository.

## Mein bester Code
Ein Code-Ausschnitt, den du selbst geschrieben hast, und was daran gut ist.

## Selbsteinschätzung
| Ich kann … | 1 | 2 | 3 | 4 | Beleg (Link) |
|------------|---|---|---|---|--------------|
| einfache Algorithmen in Python umsetzen (Verzweigung, Schleife, Funktion, Liste) | | | | |
| |
| ein Board mit Sensor und Aktor in Betrieb nehmen | | | | | |
| das Verhalten eines interaktiven Systems durch Variation von Eingaben und Rückmeldungen
untersuchen und erklären | | | | | |
| ein Gerät mit einem Zustandsdiagramm und einem Ablaufplan modellieren | | | | | |

1 = noch nicht · 2 = mit Hilfe · 3 = selbstständig · 4 = kann es anderen erklären

## Retrospektive
Die Vorsätze eures Teams für P2, und ein persönlicher Vorsatz.

## Feedback
Das Feedback, das ihr bei der Präsentation bekommen habt, in einem Satz,
und was du davon hältst.
```

Checkliste vor dem letzten Commit:

- ☐ `p1/woche-01.md` bis `p1/woche-07.md` sind da.
- ☐ Journalaufträge 1–7 sind beantwortet (Überschriften aus den Vorlagen).
- ☐ Bilder liegen in `images/` und werden angezeigt (Vorschau prüfen).
- ☐ `README.md` verlinkt alle P1-Einträge.
- ☐ Keine Passwörter, keine Fotos von Personen ohne Zustimmung.

Commit-Nachricht: `P1 Woche 7: Projektrückblick, Abgabe P1`.

5. Abschlussquiz P1 (5 min)

Abschlussquiz · P1 Code & Gadget

1. Was gibt dieses Programm aus?

```python
werte = [4, 8, 6]
summe = 0
for w in werte:
    if w > 5:
        summe = summe + w
print(summe)
```

a) 18

b) 14

c) 8

d) 3

2. `zahl = input("Zahl: ")`. Wie rechnest du mit der Eingabe?

a) `zahl = int(input("Zahl: "))`

b) `zahl = input(int("Zahl: "))`

c) `zahl = number(input("Zahl: "))`

d) Gar nicht, `input` liefert schon eine Zahl.

3. Ein Rollo hat die Zustände OBEN, UNTEN und FÄHRT. Welches ist ein **Ereignis**, das einen Übergang auslöst?

a) MOTOR

b) FÄHRT

c) Endschalter erreicht

d) Rollo

4. Eine Taste an einem Pin mit Pull-up soll die LED **genau einmal pro Druck** umschalten. Was braucht man?

a) Ein längeres `time.sleep`.

b) Einen zweiten Pin.

c) `taste.value() == 1` statt `== 0`.

d) Flankenerkennung: nur den Wechsel von nicht gedrückt zu gedrückt auswerten.

5. Welche Aufteilung des Codes haben wir in P1 verwendet?

a) Alles in `main.py`, damit man nichts suchen muss.

b) `hardware.py` (board-spezifisch), `logik.py` (testbar am PC), `main.py` (Hauptschleife)

c) Eine Datei pro Zustand.

d) `input.py`, `output.py`, `test.py`

6. Ihr ersetzt bei einem Wecker den Ton durch ein kurzes Aufleuchten der LED. Was untersucht ihr damit im Sinne der Mensch-Maschine-Interaktion?

a) Wie sich eine andere Rückmeldung auf das Verhalten der Nutzer:innen auswirkt (z. B. ob sie aufwachen).

b) Wie schnell der Prozessor ist.

c) Ob der Code kürzer wird.

d) Wie viel Strom die LED braucht.

7. Was ist der Zweck einer **Retrospektive**?

a) Die Noten für das Projekt festlegen.

b) Das Produkt vorführen.

c) Die Zusammenarbeit im Team verbessern, mit konkreten Vorsätzen für das nächste Projekt.

d) Den Code auf Fehler prüfen.

8. Wo steht im Kanban-Board eine Karte, deren Akzeptanzkriterien erst zum Teil erfüllt sind?

a) Fertig

b) Backlog, sie wird gelöscht.

c) Irgendwo, Hauptsache sichtbar.

d) In Arbeit

Ausblick: P2 Wetterstation

Euer Board kann jetzt messen und reagieren. In P2 lernt es **funken**: Messwerte über WLAN verschicken, speichern und auswerten. Wer mit ESP32 oder Pico W gebaut hat, verwendet das Board weiter, und `hardware.py` wächst einfach um neue Funktionen. Die Vorsätze aus der heutigen Retrospektive nehmt ihr mit.

P2 · WETTERSTATION / IOT

~7 Wochen · Themenbereiche 04, 05, 01, 11

Sensoren vernetzen, Daten erfassen → speichern → analysieren; Netzwerkgrundlagen, Energie & Nachhaltigkeit.

> 🚧 *Diese Seite ist in Arbeit.*

P3 · KI-LABOR

~7 Wochen · Themenbereiche 03, 01, 11

Entscheidungsbäume, Perzeptron, neuronale Netze, generative KI — und der EU AI Act.

> 🚧 *Diese Seite ist in Arbeit.*

P4 · DATENDETEKTIVE

~6 Wochen · Themenbereiche 01, 10

SQL & Datenmodellierung; Verschlüsselung, DSGVO-Sphären, OSINT & dein Datenfußabdruck.

> 🚧 *Diese Seite ist in Arbeit.*

P5 · WEB-PORTFOLIO

~7 Wochen · Themenbereiche 08, 07, 05

Deine agil entwickelte Website als Showcase aller Projekte; Anforderungen, client-/serverseitige Anteile.

> 🚧 *Diese Seite ist in Arbeit.*

SELBSTLERNEN

Lernpfade zum eigenständigen Erarbeiten: kuratierte Links und Videos, je ein **Journalauftrag** und ein **Quiz** pro Lernpfad.

DATENSTRUKTUREN

> 🚧 *Diese Seite ist in Arbeit.*

LAUFZEIT & ALGORITHMENANALYSE

> 🚧 *Diese Seite ist in Arbeit.*

CODE-QUALITÄT & REFACTORING

> 🚧 *Diese Seite ist in Arbeit.*

INTERAKTIONSFORMEN (MENSCH-MASCHINE)

> 🚧 *Diese Seite ist in Arbeit.*

STUFENMODELL & NETZBASIERTE DIENSTE

> 🚧 *Diese Seite ist in Arbeit.*

BERUFSFELDER & ETHIK

> 🚧 *Diese Seite ist in Arbeit.*

WERKZEUGE

Setup-Anleitungen für alle Werkzeuge des Kurses — für Schul-PCs und eigene Geräte (BYOD). Browser-Werkzeuge funktionieren überall.

PYTHON & THONNY

Thonny ist die Python-Entwicklungsumgebung des Kurses. Sie bringt Python mit, hat eine eingebaute Shell, einen Debugger für Anfänger:innen und kann später direkt mit MicroPython-Boards sprechen. Ein Werkzeug für das ganze Jahr.

Installation

Windows

1. Installer von thonny.org laden (Button „Windows“).
2. Installer ausführen, Standardeinstellungen belassen. Python ist enthalten, es muss nichts extra installiert werden.
3. Thonny aus dem Startmenü starten.

Auf Schul-PCs ist Thonny bereits installiert; falls nicht, gibt es auf thonny.org auch eine **portable Version**, die ohne Installationsrechte aus einem Ordner oder USB-Stick läuft.

macOS

1. Auf thonny.org den Mac-Download wählen (Apple Silicon oder Intel, je nach Gerät).
2. Die `.pkg`-Datei öffnen und den Anweisungen folgen.
3. Thonny aus dem Programme-Ordner starten. Beim ersten Start eventuell Rechtsklick → Öffnen, falls macOS die Herkunft prüft.

Linux

Am einfachsten über die Paketverwaltung:

```
sudo apt install thonny      # Debian / Ubuntu / Mint / Pop!_OS
```

Alternativ das Installationsskript von thonny.org verwenden.

Browser (Fallback)

Wenn nichts installiert werden kann:

- python.microbit.org — Python-Editor mit micro:bit-Simulator, läuft komplett im Browser.
- Wokwi — Simulator für Pico, ESP32 und mehr, siehe Wokwi & Browser-Simulatoren.

Für reines PC-Python ohne Board funktioniert jeder Online-Python-Editor, der `input()` unterstützt.

Erster Start

1. Beim ersten Start fragt Thonny nach Sprache: **Deutsch** wählen (später änderbar unter *Werkzeuge → Optionen → Allgemein*).
2. Oben ist der **Editor** (hier schreibst du Programme), unten die **Shell** (hier probierst du einzelne Befehle).
3. In die Shell tippen und Enter drücken:

```
print("Hallo Maker-Reise!")
```

4. Ein Programm: **Datei** → **Neu**, Code eintippen, **Datei** → **Speichern** als `test.py`, dann grüner **Play-Button** (oder **F5**).

Die wichtigsten Knöpfe

Knopf / Taste	Bedeutung
▶ Play (F5)	Aktuelle Datei ausführen
■ Stopp	Laufendes Programm abbrechen (wichtig bei `while True`)
🐞 Debug (Strg+F5)	Programm Schritt für Schritt ausführen, Variablen beobachten
Ansicht → Variablen	Zeigt alle Variablen und ihre Werte während des Programms
Ansicht → Assistent	Erklärt Fehlermeldungen in verständlicher Sprache

Wenn ein Programm nicht mehr reagiert

Stopp-Button drücken oder in die Shell klicken und **Strg+C**. Bei einer Endlosschleife ist das normal, kein Fehler.

Videos

Kurze Einführung in die Oberfläche:

▶ *Video: https://www.youtube.com/watch?v=93J6Sxk55mg*

Erste Schritte mit Python in Thonny:

▶ *Video: https://www.youtube.com/watch?v=k7CLrRkzC5o*

Später im Kurs

Thonny ist das Werkzeug für **P1 und P2**. Ab **P3** wechselt der Kurs zu **Visual Studio Code**, weil dort Notebooks, Datenanalyse, SQL und später Web-Entwicklung mit Git besser aufgehoben sind. Eine eigene Setup-Anleitung folgt zum Start von P3.

Ab P1 Woche 3 verbindest du Thonny mit deinem Board: *Werkzeuge* → *Optionen* → *Interpreter* → MicroPython auswählen. Anleitung unter MicroPython-Boards.

MICROPYTHON-BOARDS (MICRO:BIT, PICO, ESP32, ARDUINO)

MicroPython ist Python für kleine Computer ohne Bildschirm und Tastatur, sogenannte **Mikrocontroller**. Du schreibst denselben Python-Code wie am PC (`if`, `while`, Funktionen, Listen), dazu kommen Module wie `machine` oder `microbit`, mit denen du Pins, LEDs und Sensoren ansprichst. Programmiert wird in Thonny, das Board hängt per USB am PC.

Diese Seite brauchst du ab **P1 Woche 3**. Kein Board zur Hand? Derselbe Code läuft im Wokwi-Simulator.

Welches Board habe ich?

Board	Woran du es erkennst	MicroPython?
micro:bit	Taschengroß, 5×5-LED-Anzeige, Tasten A und B. **V2** hat ein goldenes Logo zum Antippen und einen Lautsprecher auf der Rückseite.	ja
Raspberry Pi Pico	Grüne Leiste, Aufdruck „Raspberry Pi Pico“ (**Pico W** / **Pico 2 W** mit silbernem WLAN-Chip-Deckel), Taste **BOOTSEL**.	ja
ESP32	Meist schwarzes Board mit silbernem Metalldeckel „ESP32-WROOM“, zwei Tasten **EN** und **BOOT**. Viele Hersteller, Pins unterschiedlich beschriftet.	ja
Arduino Nano ESP32	Kleines Arduino-Board, Aufdruck „Nano ESP32“.	ja
Arduino UNO R3 / UNO R4	Großes Arduino-Board mit Buchsenleisten.	**nein**, siehe Tab „Arduino“

Welche Stärken die Boards haben, steht in Woche 2.

Was du brauchst

- Ein **USB-Datenkabel**. Viele Kabel aus Handy-Ladegeräten können nur laden, keine Daten übertragen. Wird das Board nicht erkannt, ist das die häufigste Ursache. Markiere funktionierende Kabel mit Klebeband.
- Für Pico, ESP32 und Nano ESP32: ein **Steckbrett** (Breadboard), **Steckkabel**, LEDs, **Widerstände** 220 Ω und 10 kΩ, eine **Taste**.
- Für micro:bit: nichts, alles Nötige ist eingebaut. Für eigene Bauteile **Krokodilklemmen** oder ein **Edge-Connector-Breakout**.

Teil 1 · MicroPython aufspielen und mit Thonny verbinden

Das musst du **einmal pro Board** machen. Danach genügt: Board anstecken, Thonny starten.

micro:bit

Mit Thonny:

1. micro:bit per USB anschließen. Am PC erscheint ein Laufwerk **MICROBIT**.
2. Thonny: **Werkzeuge → Optionen → Interpreter → MicroPython (BBC micro:bit)**.
3. Unten rechts **Install or update MicroPython** klicken, das Laufwerk MICROBIT wählen, **Install**. Die gelbe LED auf der Rückseite blinkt, dann startet das Board neu.
4. **OK**. In der Shell erscheint `>>>`. Test:

```
from microbit import *
display.scroll("Hallo")
```

Ohne Installation (Browser): python.microbit.org öffnen, Code schreiben, mit **Send to micro:bit** übertragen (Chrome oder Edge, verbindet sich direkt über USB). In anderen Browsern **Save** klicken und die `.hex`-Datei auf das Laufwerk MICROBIT ziehen. Der Editor hat einen eingebauten Simulator, siehe Wokwi & Browser-Simulatoren.

Raspberry Pi Pico

1. Taste **BOOTSEL** gedrückt halten, **dann** das USB-Kabel einstecken, dann loslassen. Am PC erscheint ein Laufwerk **RPI-RP2** (Pico 2: **RP2350**).
2. Thonny: **Werkzeuge → Optionen → Interpreter → MicroPython (Raspberry Pi Pico)**.
3. Unten rechts **Install or update MicroPython** klicken.
4. **Target volume:** das Laufwerk RPI-RP2. **Variant:** genau dein Modell wählen: *Pico / Pico H*, *Pico W / Pico WH*, *Pico 2* oder *Pico 2 W*. Die falsche Variante läuft, aber ohne WLAN oder gar nicht.
5. **Install**. Nach dem Kopieren verschwindet das Laufwerk, das Board startet neu.
6. Fenster schließen, **OK**. In der Shell erscheint `>>>`. Test:

```
from machine import Pin
Pin("LED", Pin.OUT).value(1)    # Onboard-LED leuchtet
```

Treiber sind unter Windows 10/11, macOS und Linux **nicht** nötig.

ESP32

1. ESP32 per USB anschließen.
2. **Treiber prüfen (nur Windows, nur beim ersten Mal):** Im Geräte-Manager unter *Anschlüsse (COM & LPT)* muss ein neuer **COM-Port** erscheinen. Fehlt er, auf den kleinen Chip neben der USB-Buchse schauen und den Treiber installieren:
   - Aufdruck **CP210x** (Silicon Labs) → CP210x-Treiber
   - Aufdruck **CH340** / **CH9102** (WCH) → CH340-Treiber
3. Thonny: **Werkzeuge → Optionen → Interpreter → MicroPython (ESP32)**, bei *Port* den neuen Port wählen (`COM3`, `/dev/ttyUSB0`, `/dev/cu.usbserial-…`).
4. Unten rechts **Install or update MicroPython (esptool)** klicken.
5. **Target port:** dein Port. **MicroPython family:** *ESP32*. **Variant:** *Espressif • ESP32 / WROOM*. Hast du ein ESP32-**S3** oder **C3**, die passende Familie wählen.
6. **Install**. Bleibt es bei `Connecting...` hängen: Taste **BOOT** gedrückt halten, bis Prozentzahlen erscheinen.
7. Fenster schließen, **OK**. In der Shell erscheint `>>>`. Test:

```
from machine import Pin
Pin(2, Pin.OUT).value(1)        # Onboard-LED (bei den meisten DevKits)
```

Arduino

Arduino UNO R3 und UNO R4: Für diese Boards gibt es **kein offizielles MicroPython**. Der Kurs ist in Python, darum gibt es zwei Wege:

1. **Simulator:** In Wokwi ein **ESP32**-Projekt anlegen und den ESP32-Code des Kurses verwenden. Das ist gleichwertig: Ihr baut dieselbe Schaltung virtuell auf.
2. **Board tauschen:** ESP32, Pico oder **Arduino Nano ESP32** bei der Lehrkraft holen.

Arduino Nano ESP32:

1. Arduino MicroPython Installer laden und starten.
2. Nano ESP32 anschließen. Erkennt der Installer das Board nicht: **Reset**-Taste zweimal schnell hintereinander drücken.
3. Board auswählen, **Install MicroPython**.
4. Thonny: **Werkzeuge → Optionen → Interpreter → MicroPython (ESP32)**, Port wählen, **OK**. Test:

```
from machine import Pin
Pin(48, Pin.OUT).value(1)       # Onboard-LED (D13)
```

Code wie im ESP32-Tab, aber mit **anderen Pinnummern**, siehe Teil 3.

Teil 2 · Arbeiten mit dem Board in Thonny

Sobald Thonny mit dem Board verbunden ist, gilt:

- Die **Shell** spricht mit dem **Board**. `print(1 + 1)` rechnet der Mikrocontroller.
- **▶ Play (F5)** schickt die Datei im Editor zum Board und führt sie dort aus. Sie wird dabei **nicht** gespeichert: Nach dem Abstecken ist sie weg.
- **■ Stopp** (oder **Strg+C** in der Shell) bricht das Programm ab. Ein zweiter Klick auf Stopp startet das Board neu (Soft-Reset).

Dateien auf dem Board

Ansicht → Dateien zeigt oben die Dateien auf deinem PC, unten (Überschrift **MicroPython device**) die Dateien auf dem Board.

Was	Wie
Datei aufs Board speichern	**Datei → Speichern unter → MicroPython device**, Namen eintippen
Datei vom PC hochladen	Oben im Dateien-Bereich (PC) Rechtsklick auf die Datei → **Upload nach /**
Datei vom Board holen	Rechtsklick auf die Datei auf dem Gerät → **Download nach ...**
Datei löschen	Rechtsklick → **Löschen**

Zwei Dateinamen haben eine besondere Bedeutung:

Datei	Bedeutung
`main.py`	Startet automatisch, sobald das Board Strom bekommt, auch an einer Powerbank. Das ist euer Gadget-Programm.
`boot.py`	Startet noch vor `main.py`. Im Kurs nicht verändern.

Eigene Module wie `hardware.py` (ab Woche 4) müssen **auf dem Board** liegen, damit `import hardware` funktioniert.

Der Code gehört ins Repository, nicht nur aufs Board

Das Board ist kein Speicherort. Speichere jede Datei auch auf dem PC und lade sie ins Team-Repository hoch. Ein Board kann verloren gehen, getauscht oder neu bespielt werden.

Teil 3 · Pinbelegung und Kurs-Verkabelung

Alle Programme im Kurs verwenden dieselbe Verkabelung pro Board. Halte dich daran, dann laufen die Beispiele ohne Änderung.

Strom aus beim Stecken

- Bauteile nur stecken oder umstecken, wenn das USB-Kabel **abgezogen** ist.
- Alle Boards arbeiten mit **3,3 V**. Niemals **5 V** an einen Eingangspin legen.
- LEDs immer mit **Vorwiderstand** (220 Ω). Das **lange Bein** der LED zeigt zum Pin, das kurze zu GND.
- **3V3** und **GND** nie direkt verbinden (Kurzschluss).

micro:bit

Für P1 ist **nichts zu verkabeln**: Tasten A/B, 5×5-Anzeige, Lichtsensor (die Anzeige selbst), Beschleunigungssensor, Temperatur, Mikrofon und Lautsprecher (V2) sind eingebaut.

Kurs-Funktion	micro:bit
Taste	`button_a`, `button_b`
LED	Pixel auf `display`
Summer	eingebauter Lautsprecher, `music.pitch(...)`
Lichtsensor	`display.read_light_level()` (0–255)
Lage	`accelerometer.get_x()`

Eigene Bauteile schließt du mit Krokodilklemmen an die großen Ringe **0**, **1**, **2**, **3V** und **GND** an:

```
from microbit import *

pin0.write_digital(1)        # externe LED an Pin 0 (mit 220 Ω) ein
wert = pin1.read_analog()    # Analogwert an Pin 1, 0–1023
print(wert)
```

Die kleinen Pins am Rand erreichst du nur mit einem Breakout-Board. Viele davon gehören zur LED-Anzeige. Bleib bei 0, 1 und 2.

Raspberry Pi Pico

Achtung: Die Zahl im Code ist die **GP-Nummer**, nicht die Nummer des Beinchens. `Pin(15)` ist **GP15**, das ist Beinchen 20.

Kurs-Funktion	GP-Pin	Beinchen	Anschluss
Taste	**GP14**	19	Taste zwischen GP14 und GND (Beinchen 18)
LED	**GP15**	20	GP15 → 220 Ω → LED langes Bein, kurzes Bein → GND
Onboard-LED	`"LED"`	–	eingebaut, keine Verkabelung
Summer (passiv)	**GP16**	21	GP16 → Summer → GND (Beinchen 23)
Lichtsensor / Poti	**GP26** (ADC0)	31	3V3 → LDR → GP26 → 10 kΩ → GND
3,3 V	3V3(OUT)	36	Plus für Sensoren
GND	GND	3, 8, 13, 18, 23, 28, 33, 38	jeder GND ist gleich

Analoge Eingänge gibt es nur an **GP26**, **GP27** und **GP28**. Offizielles Pinout zum Ausdrucken: Pico, Pico W.

Das Steckbrett: Die Pico sitzt über der Mittelrille, jede Fünferreihe neben einem Beinchen ist mit diesem verbunden. Die langen Leisten am Rand (rot/blau) eignen sich für 3V3 und GND.

ESP32

Gilt für das verbreitete **ESP32-DevKit** (30 oder 38 Pins). Die Zahl im Code ist die **GPIO-Nummer**, auf dem Board meist als `D18`, `G18`, `IO18` oder nur `18` aufgedruckt.

Kurs-Funktion	GPIO	Anschluss
Taste	**4**	Taste zwischen GPIO 4 und GND
LED	**18**	GPIO 18 → 220 Ω → LED langes Bein, kurzes Bein → GND
Onboard-LED	**2**	eingebaut bei den meisten DevKits (nicht bei allen)
Summer (passiv)	**19**	GPIO 19 → Summer → GND

Kurs-Funktion	GPIO	Anschluss
Lichtsensor / Poti	**34**	3V3 → LDR → GPIO 34 → 10 kΩ → GND

Pins mit Haken:

Pins	Hinweis
6–11	**Nie verwenden**, gehören zum internen Speicher.
0, 2, 12, 15	Beeinflussen den Start. Nur verwenden, wenn nötig (GPIO 2 für die Onboard-LED ist in Ordnung).
34, 35, 36 (VP), 39 (VN)	**Nur Eingang**, kein `Pin.PULL_UP`. Gut für Sensoren, schlecht für Tasten.
32–39	Analoge Eingänge, die auch mit **WLAN** funktionieren (wichtig für P2). Andere ADC-Pins (z. B. 25, 26, 27) gehen nicht, solange WLAN aktiv ist.

Analog-Eingänge immer mit `adc.atten(ADC.ATTN_11DB)` einstellen, sonst endet der Messbereich bei ca. 1 V.

Arduino

Arduino Nano ESP32: Auf dem Board stehen Arduino-Namen (`D2`, `A0`), MicroPython verwendet aber die **GPIO-Nummern** des ESP32-S3. Übersetzung:

Arduino	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12	D13
GPIO	44	43	5	6	7	8	9	10	17	18	21	38	47	48

Arduino	A0	A1	A2	A3	A4	A5	A6	A7
GPIO	1	2	3	4	11	12	13	14

Kurs-Verkabelung (so ändert sich am ESP32-Code möglichst wenig):

Kurs-Funktion	Anschluss am Nano	im Code	Unterschied zum ESP32-Code
Taste	**A3** ↔ GND	`Pin(4, …)`	keiner
LED	**D9** → 220 Ω → LED → GND	`Pin(18, …)`	keiner
Onboard-LED	D13	`Pin(48, …)`	statt 2
Summer	**D8** → Summer → GND	`PWM(Pin(17))`	statt 19
Lichtsensor / Poti	3V3 → LDR → **A0** → 10 kΩ → GND	`ADC(Pin(1))`	statt 34

In `hardware.py` (Woche 4) änderst du also nur die Zahlen 19 → 17 und 34 → 1. Das Board arbeitet mit **3,3 V**, nicht mit 5 V wie der UNO.

UNO R3 / UNO R4: kein MicroPython, siehe Teil 1.

Taste richtig stecken

Eine Taste mit vier Beinchen verbindet immer zwei Paare. Sicher funktioniert es, wenn du **zwei diagonal gegenüberliegende** Beinchen verwendest. Die Taste sitzt über der Mittelrille des Steckbretts.

Teil 4 · Weitere Bauteile

Für euer Gadget braucht ihr vielleicht mehr als LED, Taste und Lichtsensor. Jedes Bauteil wird in `hardware.py` zu **einer kleinen Funktion**, der Rest des Codes bleibt gleich (siehe Woche 4).

Vorgeschlagene Pins, die sich mit der Kurs-Verkabelung nicht überschneiden:

Bauteil	Pico	ESP32	Nano ESP32	Strom
Bewegungsmelder (PIR, HC-SR501)	GP17	27	D4 → 7	5 V
Ultraschall-Abstand: Trig	GP18	25	D5 → 8	5 V (HC-SR04) / 3,3 V (HC-SR04**P**)
Ultraschall-Abstand: Echo	GP19	26	D6 → 9	siehe Warnung
Servo (SG90)	GP20	13	D7 → 10	5 V
Temperatur/Feuchte (DHT11/DHT22)	GP21	33	D10 → 21	3,3 V
OLED-Display (I²C): SDA / SCL	GP4 / GP5	21 / 22	A4 → 11 / A5 → 12	3,3 V
zweiter Analogwert	GP27	35	A1 → 2	3,3 V

5 V gibt es am Pin **VBUS** (Pico, Beinchen 40), **5V/VIN** (ESP32) bzw. **VBUS** (Nano ESP32), solange das Board per USB versorgt wird.

In den Beispielen unten stehen die **Pico**-Pins. Für andere Boards die Zahl aus der Tabelle einsetzen. Auf dem micro:bit sind die meisten dieser Bauteile umständlich (wenige Pins, keine 5 V). Dort besser die eingebauten Sensoren nutzen.

Bewegungsmelder (PIR)

Der PIR liefert 1, solange er Bewegung erkennt. Mit den zwei Drehreglern stellst du Empfindlichkeit und Haltezeit ein. Nach dem Einschalten braucht er etwa eine Minute zum Kalibrieren.

```
from machine import Pin

_pir = Pin(17, Pin.IN)


def bewegung():
    """True, solange Bewegung erkannt wird."""
    return _pir.value() == 1
```

Ultraschall-Abstandssensor (HC-SR04)

```
from machine import Pin, time_pulse_us
import time

_trig = Pin(18, Pin.OUT)
_echo = Pin(19, Pin.IN)


def abstand_cm():
    """Abstand in cm, oder -1 wenn nichts gemessen wurde."""
    _trig.value(0)
    time.sleep_us(2)
    _trig.value(1)
    time.sleep_us(10)
    _trig.value(0)
    dauer = time_pulse_us(_echo, 1, 30000)   # höchstens 30 ms warten
    if dauer < 0:
        return -1
    return dauer * 0.0343 / 2                # Schall: 343 m/s, hin und zurück
```

Servo (SG90)

Braun = GND, Rot = **5 V**, Orange = Signal. Ein Servo zieht viel Strom: Startet das Board neu, sobald sich der Servo bewegt, reicht die USB-Versorgung nicht. Dann den Servo über eine eigene Stromquelle versorgen (GND verbinden).

```
from machine import Pin, PWM

_servo = PWM(Pin(20))
_servo.freq(50)


def servo(winkel):
    """Dreht den Servo auf 0–180 Grad."""
```

```
    # 0,5 ms (0°) bis 2,5 ms (180°) von 20 ms
    _servo.duty_u16(1638 + winkel * (8192 - 1638) // 180)
```

Temperatur und Luftfeuchte (DHT11 / DHT22)

Das Modul `dht` ist bei Pico und ESP32 schon dabei. Der Sensor darf höchstens **einmal pro Sekunde** (DHT22: alle 2 s) abgefragt werden.

```
from machine import Pin
import dht

_dht = dht.DHT11(Pin(21))      # beim DHT22: dht.DHT22(...)


def temperatur():
    _dht.measure()
    return _dht.temperature()   # °C


def feuchte():
    _dht.measure()
    return _dht.humidity()      # %
```

OLED-Display (SSD1306, 128×64, I²C)

Den Treiber einmal aufs Board installieren: Thonny **Werkzeuge → Verwalte Pakete …**, nach `ssd1306` suchen, **Installieren**.

```
from machine import Pin, I2C
import ssd1306

_i2c = I2C(0, sda=Pin(4), scl=Pin(5))     # ESP32: I2C(0, sda=Pin(21), scl=Pin(22))
_oled = ssd1306.SSD1306_I2C(128, 64, _i2c)


def anzeigen_text(zeile1, zeile2=""):
    _oled.fill(0)
    _oled.text(zeile1, 0, 0)
    _oled.text(zeile2, 0, 16)
    _oled.show()
```

Meldet `I2C(...)` einen Fehler oder bleibt das Display dunkel: `print(_i2c.scan())` muss `[60]` ausgeben. Eine leere Liste heißt: SDA/SCL vertauscht oder nicht verbunden.

Fehlerbehebung

Problem	Wahrscheinliche Ursache	Lösung
Thonny findet kein Gerät, kein Port in der Liste	Ladekabel statt Datenkabel, Treiber fehlt (ESP32)	anderes Kabel, anderer USB-Anschluss, Treiber aus Teil 1
`Device is busy or does not respond`	`main.py` läuft in einer Endlosschleife	mehrmals **Stopp** drücken. Hilft das nicht: abstecken, einstecken, sofort Stopp
`ImportError: no module named 'hardware'`	`hardware.py` liegt nur am PC	Datei **auf das Board** speichern (Teil 2)
`ValueError: invalid pin`	Beinchen-Nummer statt GP-/GPIO-Nummer	Pinout prüfen (Teil 3)
LED bleibt dunkel	LED verkehrt herum, falscher Pin, GND fehlt	langes Bein zum Pin, Verkabelung mit Tabelle vergleichen
Taste liefert immer 0 oder immer 1	`Pin.PULL_UP` fehlt, Taste falsch gedreht, Pin 34–39 am ESP32	Pull-up setzen, diagonale Beinchen, anderen Pin
Analogwert immer 0 oder immer Maximum	falscher Pin, kein ADC-Pin, `atten` fehlt (ESP32)	Pico: GP26–28. ESP32: 32–39 mit `ATTN_11DB`

Problem	Wahrscheinliche Ursache	Lösung
Board startet neu, wenn Summer/Servo/Motor anläuft	USB liefert zu wenig Strom	eigene Stromversorgung für den Verbraucher
Programm läuft nicht ohne PC	Datei heißt nicht `main.py` oder liegt nur am PC	als `main.py` aufs Board speichern
micro:bit: Laufwerk heißt **MAINTENANCE**	Reset beim Einstecken gedrückt	abstecken, ohne Taste wieder einstecken

Wenn gar nichts mehr geht

MicroPython einfach **neu aufspielen** (Teil 1). Das löscht alle Dateien auf dem Board. Darum gehört der Code ins Repository.

Weiterlesen

- MicroPython-Dokumentation (englisch): *Quick reference* für RP2/Pico und ESP32
- micro:bit-MicroPython-Dokumentation
- Arduino Nano ESP32 und MicroPython

WOKWI & BROWSER-SIMULATOREN

Ein **Simulator** zeigt im Browser ein virtuelles Board mit LEDs, Tasten und Sensoren. Dein MicroPython-Code läuft darin **genauso wie auf dem echten Board**. Du brauchst nichts zu installieren, nur einen Browser.

Simulator	Boards	Wofür
Wokwi	Raspberry Pi Pico, ESP32	eigene Schaltungen mit Steckbrett-Bauteilen, `hardware.py`, später WLAN (P2)
python.microbit.org	micro:bit	Editor und Simulator in einem, Sensoren per Schieberegler

Wann simulieren?

Der Simulator ist **kein Notbehelf**, sondern ein gleichwertiges Werkzeug:

- **Kein Board da:** zu Hause, bei Krankheit, wenn das Team-Board gerade bei jemand anderem liegt.
- **Arduino UNO R3 / R4:** Diese Boards können kein MicroPython. Euer Team baut die Schaltung in Wokwi mit einem **ESP32** auf (siehe MicroPython-Boards).
- **Vor dem Verkabeln:** Schaltung erst virtuell aufbauen und testen. Was im Simulator nicht geht, geht auf dem Steckbrett auch nicht.
- **Zum Teilen:** Ein Wokwi-Link zeigt Code **und** Schaltung. Ideal fürs Team-Repository und für Fragen an die Lehrkraft.

Teil 1 · Wokwi: Projekt starten

1. Leeres MicroPython-Projekt öffnen:
   - Neues Pico-Projekt (MicroPython)
   - Neues ESP32-Projekt (MicroPython)
2. **Links** ist der Code (`main.py`) und die Schaltungsbeschreibung (`diagram.json`), **rechts** die Simulation.
3. Den grünen **▶-Knopf** über der Simulation drücken. Das Programm startet, Ausgaben von `print` erscheinen in der Konsole unten rechts.
4. ■ beendet die Simulation. Änderst du den Code, startet ▶ ihn neu.

Probier es aus, ohne etwas zu verkabeln (Pico, Onboard-LED):

```
from machine import Pin
import time

led = Pin("LED", Pin.OUT)       # funktioniert "LED" nicht: Pin(25, Pin.OUT)

for i in range(10):
    led.value(not led.value())
    time.sleep(0.5)
print("Fertig")
```

Das ESP32-Board in Wokwi (DevKitC) hat **keine Onboard-LED**. Dort gleich eine externe LED an GPIO 18 anschließen, siehe Teil 3.

REPL in Wokwi

Ist `main.py` fertig, erscheint in der Konsole `>>>`. Dort kannst du wie in der Thonny-Shell einzelne Befehle ausprobieren. Läuft eine Endlosschleife, klick in die Konsole und drück **Strg+C**.

Teil 2 · Bauteile hinzufügen und verdrahten

Was	Wie
Bauteil hinzufügen	Blauer **+**-Knopf über der Simulation, Bauteil suchen (z. B. *LED*, *Resistor*, *Pushbutton*, *Buzzer*, *Potentiometer*, *Photoresistor*)
Bauteil verschieben	ziehen
Bauteil drehen	anklicken, **R** drücken
Eigenschaften (Farbe, Widerstandswert)	Bauteil anklicken, im Fenster ändern
Kabel ziehen	auf ein Beinchen klicken, dann auf das Ziel-Beinchen. Zwischenklicks setzen Knickpunkte.
Kabelfarbe	Kabel anklicken, Farbe wählen. Abmachung: **Schwarz = GND**, **Rot = 3V3**
Löschen	anklicken, **Entf**

Fährst du mit der Maus über ein Beinchen, zeigt Wokwi seinen Namen (`GP15`, `GND.3`, `A`, `C`). Bei der LED ist **A** (Anode) das lange Bein (+), **C** (Kathode) das kurze.

Wokwi zeichnet nur, was in `diagram.json` steht. Mit dem Maus-Editor und mit dem Text in `diagram.json` änderst du also dasselbe.

Teil 3 · Die Kurs-Verkabelung in Wokwi

Baue die Schaltung aus Woche 3 und 4 nach. Dann läuft `hardware.py` aus dem Kurs **ohne Änderung**.

Raspberry Pi Pico

Bauteil	Verbindung
Taste	GP14 ↔ GND
LED + Widerstand 220 Ω	GP15 → Widerstand → LED **A**, LED **C** → GND
Summer (Buzzer)	GP16 → Buzzer **2** (+), Buzzer **1** (−) → GND
Potentiometer (statt Lichtsensor)	**VCC** → 3V3, **SIG** → GP26, **GND** → GND

Abkürzung über `diagram.json`: Öffne den Reiter `diagram.json`. Füge die folgenden Zeilen **direkt nach** `"parts": [` ein (vor dem Eintrag des Boards):

```
    { "type": "wokwi-pushbutton", "id": "btn1", "top": 0, "left": 150, "attrs": { "color":
"green" } },
    { "type": "wokwi-resistor", "id": "r1", "top": 80, "left": 150, "attrs": { "value":
"220" } },
    { "type": "wokwi-led", "id": "led1", "top": 50, "left": 250, "attrs": { "color": "red"
} },
    { "type": "wokwi-buzzer", "id": "bz1", "top": 140, "left": 150, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot1", "top": 240, "left": 150, "attrs": {} },
```

Und diese Zeilen **direkt nach** `"connections": [`:

```
    [ "pico:GP14", "btn1:1.l", "green", [] ],
    [ "btn1:2.l", "pico:GND.4", "black", [] ],
    [ "pico:GP15", "r1:1", "orange", [] ],
    [ "r1:2", "led1:A", "orange", [] ],
    [ "led1:C", "pico:GND.5", "black", [] ],
    [ "pico:GP16", "bz1:2", "purple", [] ],
    [ "bz1:1", "pico:GND.6", "black", [] ],
    [ "pot1:VCC", "pico:3V3", "red", [] ],
    [ "pot1:SIG", "pico:GP26", "blue", [] ],
    [ "pot1:GND", "pico:GND.8", "black", [] ],
```

Danach zurück zur Simulation und die Bauteile mit der Maus ordentlich hinschieben.

ESP32

Bauteil	Verbindung
Taste	GPIO 4 ↔ GND
LED + Widerstand 220 Ω	GPIO 18 → Widerstand → LED **A**, LED **C** → GND
Summer (Buzzer)	GPIO 19 → Buzzer **2** (+), Buzzer **1** (–) → GND
Potentiometer (statt Lichtsensor)	**VCC** → 3V3, **SIG** → GPIO 34, **GND** → GND

Abkürzung über `diagram.json`: Öffne den Reiter `diagram.json`. Füge die folgenden Zeilen **direkt nach** `"parts": [` ein (vor dem Eintrag des Boards):

```
    { "type": "wokwi-pushbutton", "id": "btn1", "top": 0, "left": 200, "attrs": { "color":
"green" } },
    { "type": "wokwi-resistor", "id": "r1", "top": 80, "left": 200, "attrs": { "value":
"220" } },
    { "type": "wokwi-led", "id": "led1", "top": 50, "left": 300, "attrs": { "color": "red"
} },
    { "type": "wokwi-buzzer", "id": "bz1", "top": 140, "left": 200, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot1", "top": 240, "left": 200, "attrs": {} },
```

Und diese Zeilen **direkt nach** `"connections": [`:

```
    [ "esp:4", "btn1:1.l", "green", [] ],
    [ "btn1:2.l", "esp:GND.2", "black", [] ],
    [ "esp:18", "r1:1", "orange", [] ],
    [ "r1:2", "led1:A", "orange", [] ],
    [ "led1:C", "esp:GND.2", "black", [] ],
    [ "esp:19", "bz1:2", "purple", [] ],
    [ "bz1:1", "esp:GND.3", "black", [] ],
    [ "pot1:VCC", "esp:3V3", "red", [] ],
    [ "pot1:SIG", "esp:34", "blue", [] ],
    [ "pot1:GND", "esp:GND.1", "black", [] ],
```

Danach zurück zur Simulation und die Bauteile mit der Maus ordentlich hinschieben.

Arduino-Teams (UNO R3 / R4) verwenden genau diese ESP32-Schaltung.

⚠ Fehler in diagram.json

Jeder Eintrag endet mit einem Komma, **außer dem letzten** in der Liste. War die Liste vorher leer (`[]`), lösch das Komma nach der letzten eingefügten Zeile. Wokwi unterstreicht Fehler rot. Heißt das Board in deiner Datei nicht `pico` bzw. `esp` (siehe `"id"` beim Board-Eintrag), ersetze den Namen in den Verbindungen.

Lichtsensor statt Potentiometer

Wokwi hat ein Lichtsensor-Modul (*Photoresistor sensor*, Anschlüsse **VCC** → 3V3, **GND** → GND, **AO** → GP26 bzw. GPIO 34). Achtung: Bei diesem Modul **sinkt** der Wert, wenn es heller wird, also genau umgekehrt wie bei der Kurs-Schaltung. In `hardware.py` dann `100 - …` rechnen:

```
def helligkeit():
    return 100 - _sensor.read_u16() * 100 // 65535
```

Für den Kurs ist das **Potentiometer** einfacher: drehen = heller/dunkler.

Teil 4 · Mehrere Dateien: `hardware.py` und `main.py`

Ab Woche 4 besteht euer Programm aus mehreren Dateien. Wokwi legt **alle Dateien** des Projekts auf das virtuelle Board, `main.py` startet automatisch.

1. Neben den Datei-Reitern auf den kleinen **Pfeil ▾** klicken → **New file…**
2. Namen eintippen, z. B. `hardware.py`, Code einfügen.
3. In `main.py` wie gewohnt `import hardware`.

Mit demselben Menü kannst du Dateien **hochladen** (*Upload file(s)…*) und das ganze Projekt als ZIP **herunterladen**.

Teil 5 · Während der Simulation

Bauteil	So bedienst du es
Taste	anklicken und gedrückt halten. **Strg**/**Cmd**+Klick hält sie dauerhaft gedrückt.
Potentiometer	am Drehknopf ziehen, oder anklicken und ← → drücken
Lichtsensor	anklicken, Schieberegler *lux* verschieben
Summer	Ton kommt aus dem Lautsprecher deines Computers (Lautstärke prüfen)

Messwerte siehst du mit `print()` in der Konsole. Einen Plotter wie in Thonny gibt es in Wokwi nicht. Für den Schwellwert in Woche 4 genügen die Zahlen in der Konsole.

Teil 6 · Speichern, teilen, abgeben

1. **Save** (oder **Strg+S**). Beim ersten Mal fragt Wokwi nach einer Anmeldung. Am einfachsten **mit GitHub anmelden**, das Konto hast du schon.
2. Nach dem Speichern hat das Projekt eine eigene Adresse, z. B. `https://wokwi.com/projects/123456789`. Diese Adresse ist dein **Link zum Teilen**.
3. Für das Team-Repository:
   - den **Wokwi-Link** in die `README.md` schreiben,
   - die Dateien (`main.py`, `hardware.py`, …) trotzdem **hochladen**, denn das Repository ist die Quelle, nicht Wokwi,
   - für das Journal einen **Screenshot** der Schaltung machen.

Wer sieht mein Projekt?

Gespeicherte Wokwi-Projekte kann jede Person sehen, die den Link kennt. Schreib keine persönlichen Daten in Code oder Projektnamen. Ein guter Projektname: `pl-gadget-<teamname>`.

Arbeitet ihr zu zweit am selben Projekt: Wokwi kann nicht gleichzeitig bearbeitet werden. Eine Person speichert, die andere öffnet den Link und klickt **Save a copy**. Zusammengeführt wird der Code im Team-Repository.

micro:bit: python.microbit.org

Der micro:bit-Python-Editor hat den Simulator gleich eingebaut.

1. python.microbit.org öffnen. Links Code schreiben.
2. Rechts den **▶-Knopf** über dem virtuellen micro:bit drücken.
3. Tasten **A** und **B** und das Logo im Bild anklicken.
4. Unter dem Simulator stellst du die **Sensoren** ein: Helligkeit, Temperatur, Lautstärke, Beschleunigung und Gesten (*shake*, *tilt*, …).
5. `print`-Ausgaben erscheinen unter dem Simulator (*Serial*).

Code für den Simulator ist **derselbe** wie für den echten micro:bit. Mit **Send to micro:bit** (Chrome/Edge) landet er direkt auf dem Board.

Speichern: Der Editor merkt sich dein Projekt im Browser, aber nur auf diesem Gerät. **Save** lädt eine `.hex`-Datei herunter (zum Weitergeben oder Flashen). Im Menü bei **Save** gibt es auch *Save Python script* für die `.py`-Datei. Die gehört ins Team-Repository.

Mehrere Dateien: Mit **Open...** eine weitere `.py`-Datei wie `hardware.py` öffnen und sie als **neue Datei zum Projekt hinzufügen**. Sie erscheint dann im Bereich *Project*.

Grenzen des Simulators

Ein Simulator ist ein **Modell** der Wirklichkeit. Einiges bildet er nicht ab:

Im Simulator	Auf dem echten Board
Sensoren liefern exakte, ruhige Werte	Werte schwanken, Schwellwerte müsst ihr am Board noch einmal prüfen
Verkehrt gesteckte LED geht einfach nicht	dasselbe, aber ein Kurzschluss kann Bauteile **zerstören**
Strom ist unbegrenzt	Servos und Motoren können das Board neu starten lassen
Zeitverhalten ungefähr echt	Reaktionsspiele und Tonlängen können sich leicht anders anfühlen
WLAN über das Netz `Wokwi-GUEST` (ohne Passwort, nur ESP32 und Pico W)	echtes Schul-WLAN, siehe P2

Darum gilt: **Entwickeln im Simulator, Endabnahme am Board.** Arduino-Teams führen ihr Gadget beim Review in Wokwi vor.

Andere Simulatoren

- **Wokwi** mit Arduino UNO: Hier programmierst du in **C++** statt Python. Nicht Teil von P1, nur für Neugierige.
- **MakeCode**: micro:bit mit Blöcken. Kennst du vielleicht aus der Unterstufe. Im Kurs schreiben wir Python.

GIT & GITHUB

GitHub ist der Ort, an dem dein Lernjournal und später der Code deiner Teams liegen. **Git** ist das Werkzeug dahinter, das jede Änderung als **Commit** festhält. In Woche 1 arbeitest du nur im Browser. Das lokale Arbeiten mit Git kommt, sobald ihr im Team Code teilt.

Teil 1 · Konto anlegen

1. github.com/signup öffnen.
2. E-Mail-Adresse (Schul-Adresse, falls vorhanden), Passwort und einen **Benutzernamen** wählen. Der Benutzername ist öffentlich sichtbar: Kein voller Klarname nötig, aber so, dass deine Lehrkraft dich zuordnen kann, z. B. `vorname-nachname-kuerzel` oder wie in der Klasse vereinbart.
3. Bestätigungscode aus der E-Mail eingeben.

⚠ Passwort

Notiere das Passwort an einem sicheren Ort. Ein vergessenes GitHub-Passwort kostet eine halbe Stunde. Wenn die Schule einen Passwort-Manager hat, verwende ihn.

Teil 2 · Lernjournal-Repository anlegen

Mit Einladungslink (Classroom 50)

Die Klasse verwendet **Classroom 50**, ein freies Werkzeug, das jeder Person ein eigenes privates Repository in der Organisation der Schule anlegt.

1. **Benutzernamen melden:** Sag deiner Lehrkraft deinen GitHub-Benutzernamen (Liste an der Tafel oder Formular). Erst danach kann sie dich freischalten.
2. classroom50.org öffnen, **Sign in with GitHub** klicken und Classroom 50 im Fenster erlauben (**Authorize**).
3. Den **Klassen-Link** der Lehrkraft öffnen (Einladung in die Organisation annehmen, falls gefragt).
4. Den **Aufgaben-Link** `lernjournal` öffnen. Die Seite zeigt deinen Benutzernamen und den Namen des Repositorys, das du bekommst.
5. **Accept assignment** klicken. Eine Checkliste zeigt den Fortschritt, das dauert bis zu einer Minute.
6. **Open repository** klicken und die Seite als **Lesezeichen** speichern.

Dein Repository heißt nach dem Muster `<klasse>-lernjournal-<benutzername>`. **Nicht umbenennen**, sonst findet Classroom 50 es nicht mehr. Es ist privat: Nur du und deine Lehrkraft sehen es. Die Ordnerstruktur und eine Vorlage sind bereits drin.

Selbst anlegen

1. Oben rechts **+** → **New repository**.
2. Repository name: `lernjournal`. Sichtbarkeit: **Private**.
3. Haken bei **Add a README file**. Dann **Create repository**.
4. Lehrkraft als Mitleserin eintragen: **Settings** → **Collaborators** → **Add people**, GitHub-Benutzernamen der Lehrkraft eingeben.
5. Ordnerstruktur anlegen: **Add file** → **Create new file**, als Dateiname `p1/woche-01.md` eintippen (der Schrägstrich erzeugt den Ordner), Inhalt aus der Vorlage einfügen, **Commit changes**.

Teil 3 · Datei im Browser bearbeiten und committen

1. Im Repository die Datei anklicken, z. B. `README.md`.
2. Rechts oben das **Stift-Symbol** (Edit this file).
3. Text ändern. Mit dem Reiter **Preview** kontrollieren, wie das Markdown aussieht.

4. Rechts oben **Commit changes…**
5. Eine kurze **Commit-Nachricht** eintragen, die sagt, was du geändert hast, z. B. `README: Name und Klasse ergänzt`. Dann **Commit changes**.

Das ist der ganze Kreislauf: **bearbeiten → Vorschau → committen**. Jeder Commit ist ein Punkt in der Zeitleiste. Unter **Commits** (oder dem Uhr-Symbol) siehst du alle bisherigen Stände und kannst jeden alten Stand ansehen.

Neue Datei anlegen

Add file → Create new file. Im Dateinamen einen Ordner mit Schrägstrich angeben: `p1/woche-02.md`.

Bild hochladen

1. **Add file → Upload files**.
2. Screenshot hineinziehen. Vorher sinnvoll benennen, z. B. `p1-w01-schalter.png`.
3. Ganz unten **Commit changes**. Das Bild liegt jetzt im Repository (lege es in den Ordner `images/`, indem du zuerst in den Ordner wechselst).
4. Im Journal-Eintrag einbinden: `![Mein Schalter](../images/p1-w01-schalter.png)`

> **Screenshot machen**
>
> Windows: **Win+Shift+S** · macOS: **Cmd+Shift+4** · Linux (GNOME): **Druck**-Taste. Das Bild landet in der Zwischenablage oder im Bilder-Ordner.

Die drei Begriffe, die du heute brauchst

Begriff	Bedeutung
Repository (Repo)	Ein Projektordner mit vollständiger Geschichte aller Änderungen.
Commit	Ein gespeicherter Stand mit Nachricht, Zeitpunkt und Autor:in. Unveränderlich.
Commit-Nachricht	Ein Satz, der sagt, was sich geändert hat. Für dich in drei Monaten und für deine Lehrkraft.

Später: Git lokal

Sobald Teams gemeinsam Code schreiben (ab P1 Sprint 1), arbeitet ihr mit Git auf dem eigenen Rechner: klonen, ändern, committen, pushen, und mit **Branches** parallel arbeiten. Diese Anleitung wird dann erweitert. Bis dahin reicht der Browser.

MARKDOWN-SPICKZETTEL

Markdown ist Text mit ein paar Sonderzeichen für die Formatierung. Dein Lernjournal ist in Markdown geschrieben, dieses Buch auch, README-Dateien auf GitHub sowieso. Du brauchst nur diese Handvoll Regeln.

Das Wichtigste

Du schreibst	Es erscheint
`# Überschrift`	große Überschrift (eine Raute = Ebene 1, `##` = Ebene 2, `###` = Ebene 3)
`**fett**`	**fett**
`*kursiv*`	*kursiv*
`- Punkt`	Aufzählung (jede Zeile mit `-` beginnen)
`1. Schritt`	nummerierte Liste
`` `code` ``	`code` im Text
`[Text](https://…)`	Link
`![Beschreibung](../images/bild.png)`	eingebettetes Bild
`---`	Trennlinie

Absätze trennst du mit einer **Leerzeile**. Ein einzelner Zeilenumbruch reicht nicht.

Code-Blöcke

Mehrzeiliger Code kommt zwischen drei Backticks, dahinter die Sprache für die Farben:

````
```python
zustand = "AUS"
print(zustand)
```
````

wird zu

```python
zustand = "AUS"
print(zustand)
```

Ein vollständiges Beispiel

```
# P1 · Woche 1 · 2026-09-14

## Gemacht
- Thonny gestartet, `hallo.py` und `schalter.py` geschrieben
- GitHub-Konto und Lernjournal angelegt

## Geklappt
- Der Schalter wechselt zwischen **AUS** und **AN**

## Hängen geblieben
- `while True` lief endlos, bis ich den Stopp-Button gefunden habe

## Belege
![Mein Schalter](../images/p1-w01-schalter.png)
```

Vorschau

Auf GitHub gibt es im Editor den Reiter **Preview**. Dort siehst du sofort, wie dein Text aussieht. In VS Code ist es später **Strg+Shift+V**.

Mehr

- Diagramme direkt im Markdown: Diagramme mit Mermaid
- Markdown-Grundlagen auf GitHub Docs (deutsch)
- CommonMark-Referenz mit interaktivem 10-Minuten-Tutorial (englisch)

DIAGRAMME MIT MERMAID

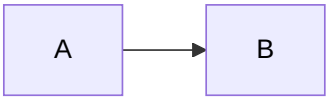
Mermaid zeichnet Diagramme aus Text. Du beschreibst in ein paar Zeilen, welche Kästchen es gibt und welche Pfeile sie verbinden, und Mermaid macht daraus eine Grafik. Weil das Diagramm nur Text ist, steht es direkt in deiner Markdown-Datei, lässt sich mit Git versionieren und ist in Sekunden geändert. Kein Zeichenprogramm, kein Screenshot, der veraltet.

So schreibst du einen Mermaid-Block

Ein Mermaid-Diagramm ist ein Code-Block mit der Sprache `mermaid`:

````
```mermaid
flowchart LR
    A --> B
```
````

wird zu



Die **erste Zeile** im Block sagt, welche Art von Diagramm du willst. Für dieses Buch brauchst du zwei: `flowchart` und `stateDiagram-v2`.

> ⚠ Wo wird Mermaid angezeigt?
>
> **GitHub** und **dieses Buch** zeigen den Block als Grafik. **Thonny** und viele Editoren zeigen nur den Text. Zum Ausprobieren nimm den Online-Editor mermaid.live: links tippen, rechts sofort das Bild sehen.

Flussdiagramm: `flowchart`

Richtung

Gleich nach `flowchart` kommt die Richtung:

Kürzel	Richtung
`LR`	von links nach rechts
`TD`	von oben nach unten

Knoten

Jeder Knoten hat eine **ID** (kurz, ohne Leerzeichen und Umlaute) und eine **Beschriftung**. Die Klammern um die Beschriftung bestimmen die Form:

Du schreibst	Form	Wofür
`A[Text]`	Rechteck	Schritt, Bauteil
`A([Text])`	abgerundet	Start, Ende
`A{Text}`	Raute	Entscheidung, Frage

Du schreibst	Form	Wofür
`A((Text))`	Kreis	das Gerät selbst

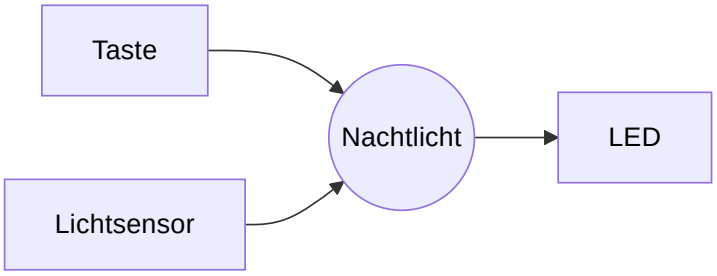
Hast du einen Knoten einmal beschriftet, reicht danach die ID.

Pfeile

Du schreibst	Bedeutung
`A --> B`	Pfeil
`A -->|ja| B`	Pfeil mit Beschriftung
`A --- B`	Linie ohne Spitze

Beispiel 1: Eingaben → Gadget → Ausgaben

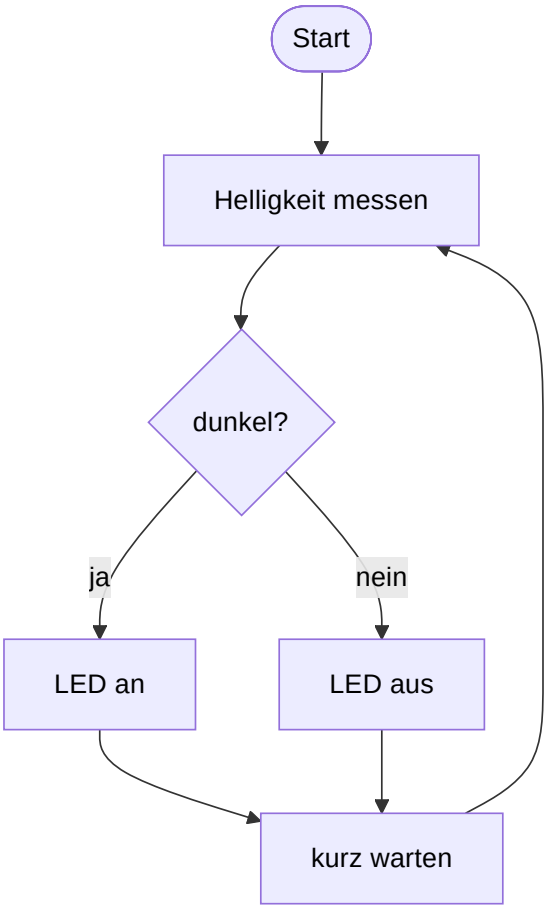
So skizziert ihr in Woche 2 euer Gadget:




```mermaid
flowchart LR
    T[Taste] --> G((Nachtlicht))
    L[Lichtsensor] --> G
    G --> LED[LED]
```

Beispiel 2: Ablaufplan mit Entscheidung

Mit einer **Raute** und beschrifteten Pfeilen zeichnest du ein `if`:




```mermaid
flowchart TD
    S([Start]) --> M[Helligkeit messen]
    M --> F{dunkel?}
    F -->|ja| AN[LED an]
    F -->|nein| AUS[LED aus]
    AN --> W[kurz warten]
    AUS --> W
    W --> M
```

Der Pfeil von `W` zurück zu `M` ist die `while True`-Schleife.

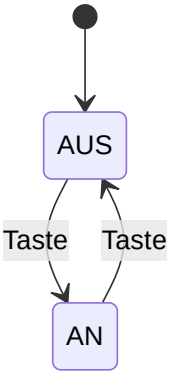
Zustandsdiagramm: `stateDiagram-v2`

Ein Zustandsdiagramm zeigt, in welchen **Zuständen** ein Gerät sein kann und welches **Ereignis** es von einem Zustand in den nächsten bringt. Mehr dazu in Woche 4.

Du schreibst	Bedeutung
`[*] --> AUS`	Startzustand ist AUS
`AUS --> AN: Taste`	Übergang von AUS nach AN beim Ereignis „Taste"
`AN --> AUS: Taste / LED aus`	Ereignis **/** was dabei passiert
`AUS --> [*]`	Endzustand (brauchst du selten)

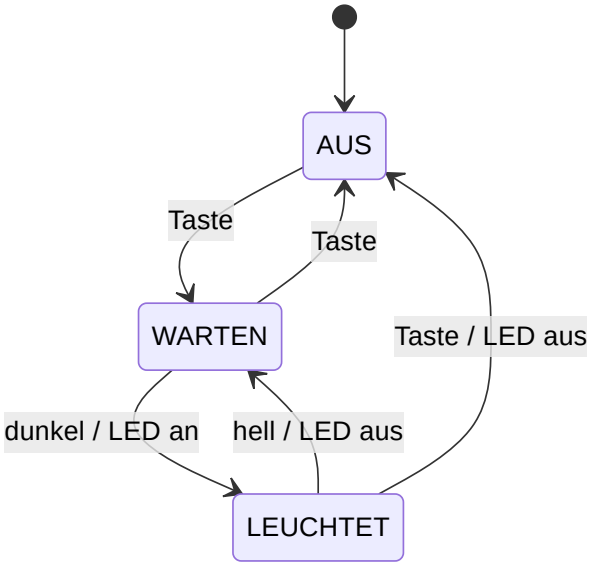
Zustände musst du nicht extra anlegen. Sobald ein Name in einem Pfeil vorkommt, gibt es ihn.

Beispiel 3: Der Schalter




```mermaid
stateDiagram-v2
    [*] --> AUS
    AUS --> AN: Taste
    AN --> AUS: Taste
```

Beispiel 4: Das Nachtlicht




```mermaid
stateDiagram-v2
    [*] --> AUS
    AUS --> WARTEN: Taste
    WARTEN --> LEUCHTET: dunkel / LED an
    LEUCHTET --> WARTEN: hell / LED aus
    WARTEN --> AUS: Taste
    LEUCHTET --> AUS: Taste / LED aus
```

Wenn das Diagramm nicht erscheint

Mermaid ist streng: Ein einziger Tippfehler, und statt der Grafik kommt eine Fehlermeldung oder gar nichts. Prüfe der Reihe nach:

- Beginnt der Block mit ```` ```mermaid ```` (klein geschrieben) und endet er mit ```` ``` ````?
- Stimmt die erste Zeile genau: `flowchart LR`, `flowchart TD` oder `stateDiagram-v2`?
- Sind die **IDs** ohne Leerzeichen, Umlaute und Sonderzeichen? `LED_AN` geht, `LED an` nicht.
- Enthält eine Beschriftung Klammern oder Anführungszeichen? Dann setze sie in Anführungszeichen: `A["Taste (rot)"]`.
- Heißt ein Knoten im Flussdiagramm `end`? Das ist ein reserviertes Wort. Nimm `Ende` oder `E([Ende])`.
- Pfeil richtig geschrieben? Es sind **zwei** Striche: `-->`.

Erst Papier, dann Mermaid

Überleg dir das Diagramm zuerst mit Stift und Papier. Mermaid ist zum sauberen Abschreiben da, nicht zum Nachdenken. Wenn es in mermaid.live funktioniert, kopierst du den Block in deine `.md`-Datei.

Mehr

- mermaid.live: Online-Editor mit Sofort-Vorschau
- Diagramme auf GitHub erstellen (deutsch)
- Mermaid-Dokumentation mit allen Diagrammarten (englisch)
